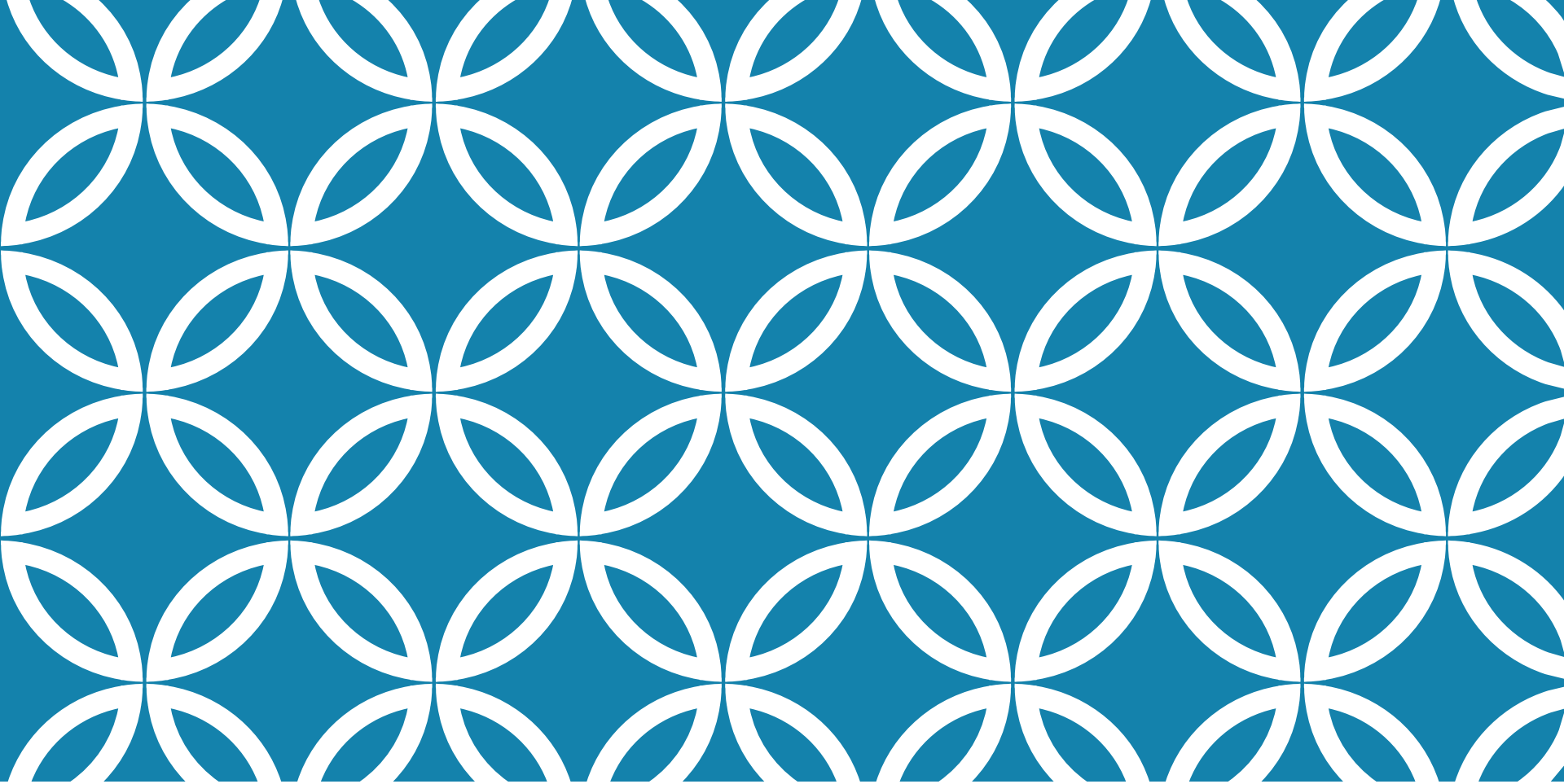


MICROKERNEL AND SERVICE-BASED ARCHITECTURES

Lecture 5

LAST TIME

Architectural Styles

- Foundations
- Layered
  - MVC and variants
- Pipeline

CONTENT

Architectural Styles

- Microkernel
- Service-based architectures
- Domain-driven design

REFERENCES

Fundamentals of Software Architecture, Mark Richards, Neal Ford, 2020 - Chapters 12, 13

Vaughn Vernon, Domain Driven Design Distilled, Addison Wesley, 2016

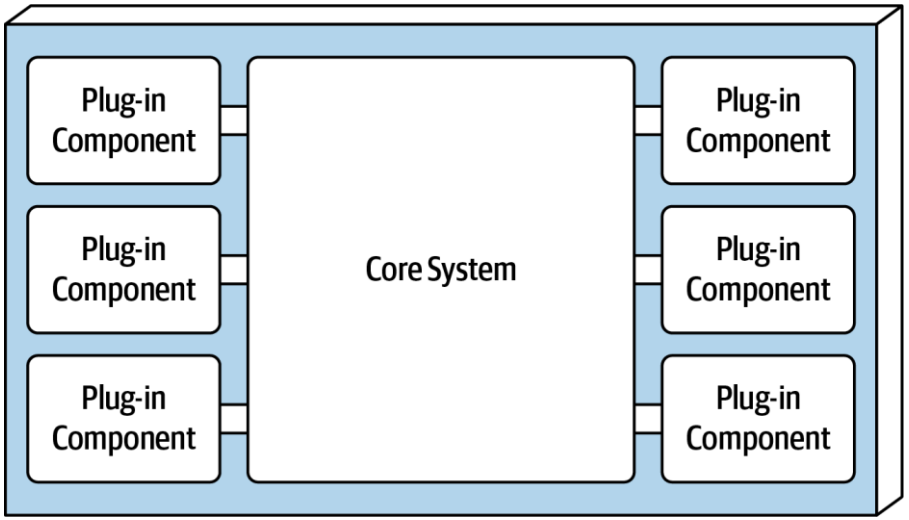
MICROKERNEL ARCHITECTURE

- Aka plug-in architecture
- Natural fit for product-based applications (single, monolithic deployment typically installed on the customer's site as a third-party product)
- Allows for customization and extension

MICROKERNEL TOPOLOGY

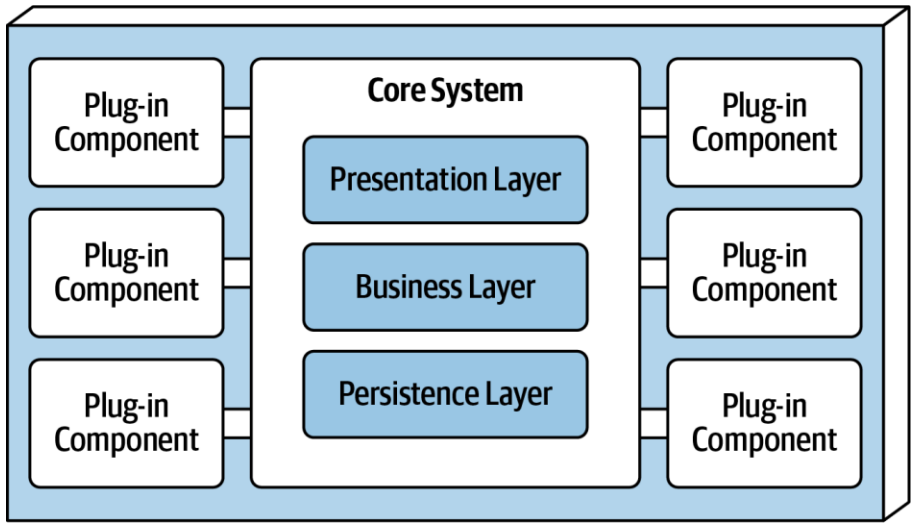
- Monolithic architecture with 2 components
  - Core system
  - Plug-in components

=> extensibility, adaptability, and isolation of application features and custom processing logic

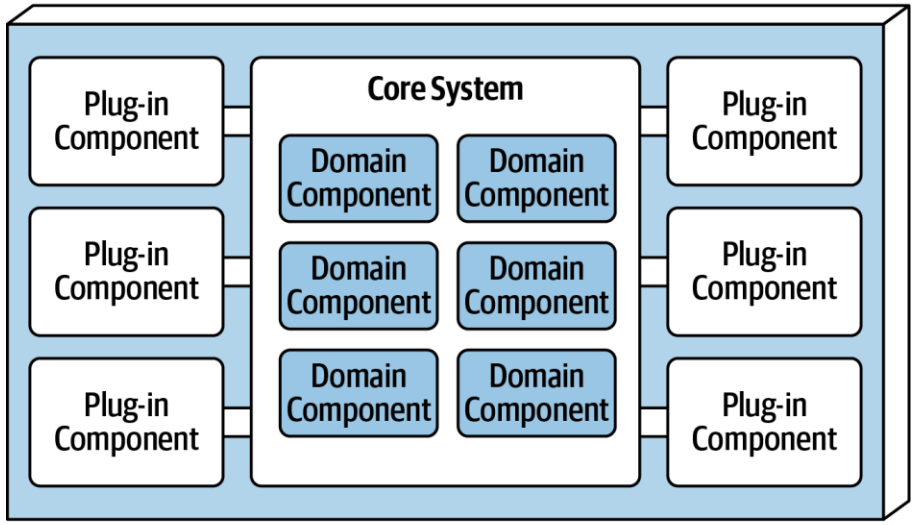


CORE SYSTEM

- The minimal functionality required to run the system
- Can be implemented as
  - Layered architecture or
  - Modular monolith
- Typically, the entire monolithic application shares a single database
- Can include the presentation layer or provide just the backend

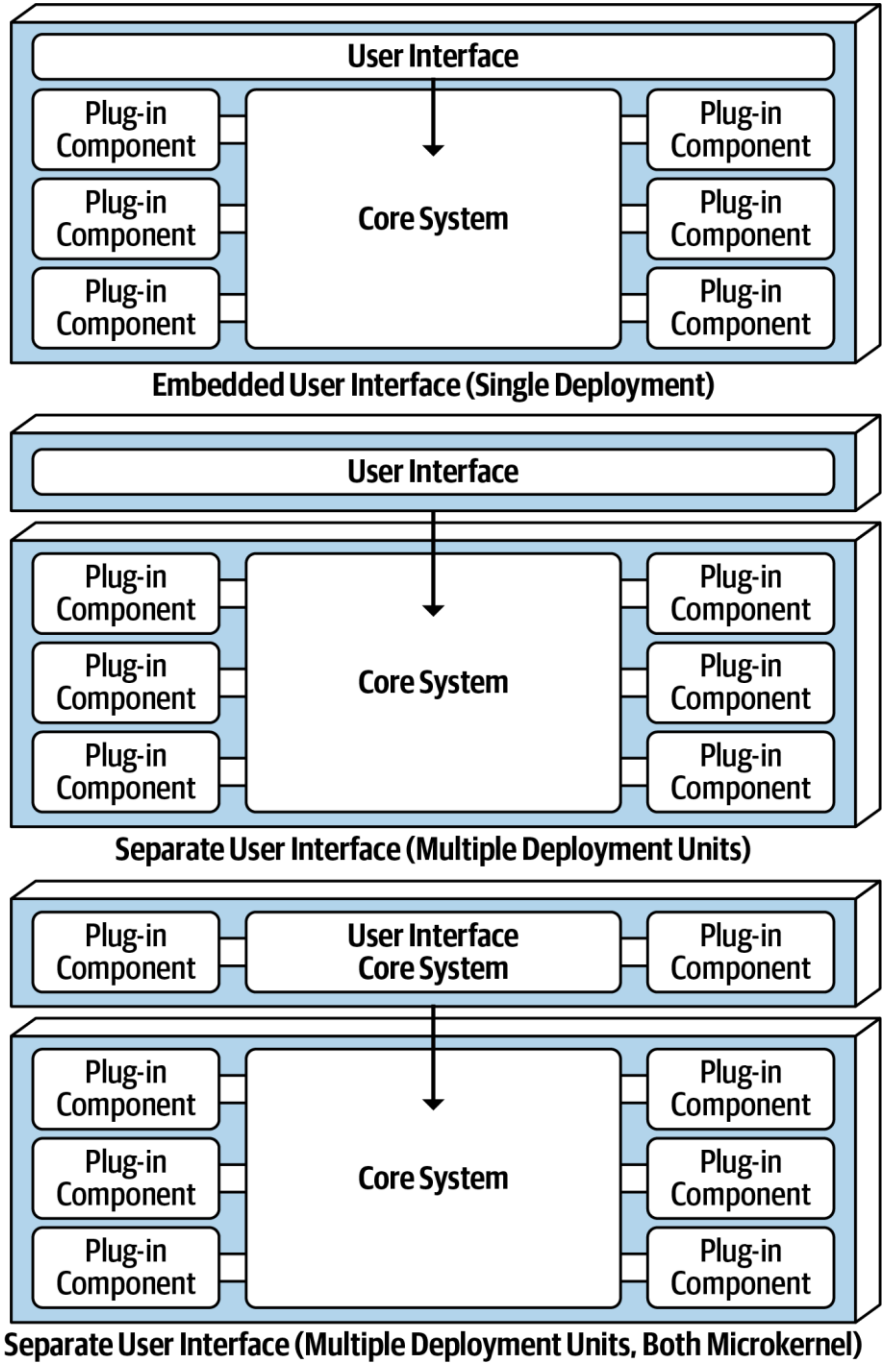


Layered Core System (Technically Partioned)



Modular Core System (Domain Partioned)

CORE SYSTEM AND UI

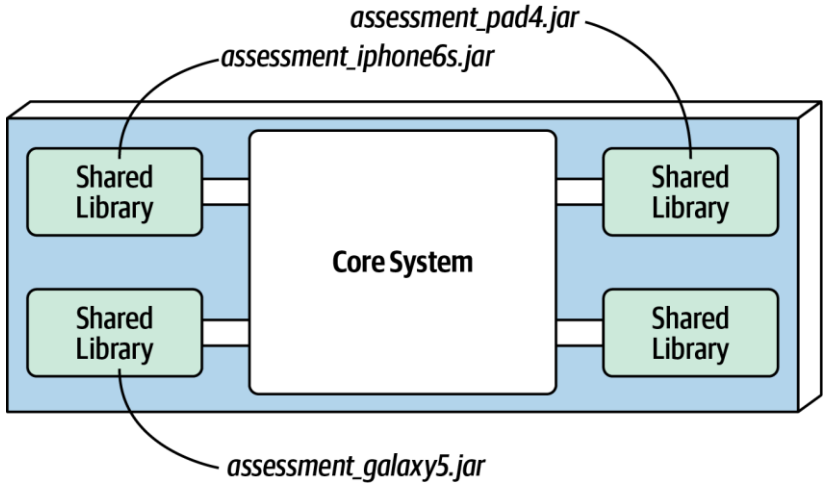


PLUG-IN COMPONENTS

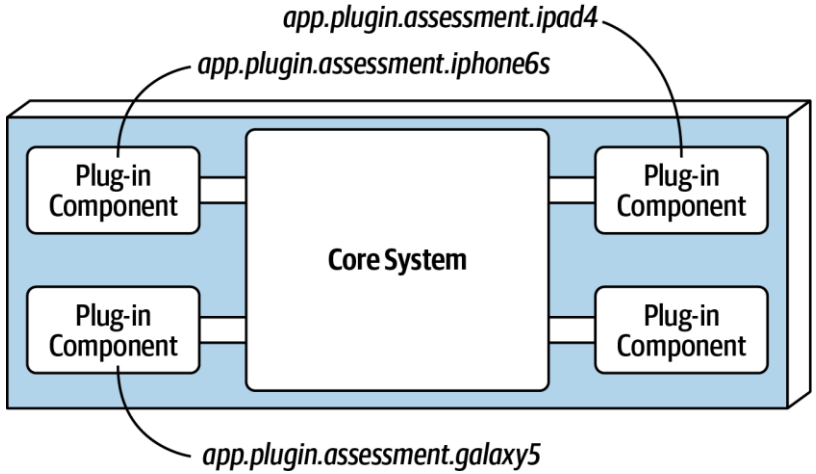
- are **standalone, independent** components that contain specialized processing, additional features, and custom code meant to enhance or extend the core system
- can be used to **isolate** highly **volatile** code => better maintainability and testability within the application.
- should be independent of each other
- communication between the plug-in components and the core system is generally point-to-point
- can be either **compile-based** or **runtime-based** (managed through frameworks ex. Open Service Gateway Initiative (OSGi) for Java, Penrose (Java), Jigsaw (Java), or Prism (.NET))

POINT-TO-POINT PLUG-IN COMPONENTS

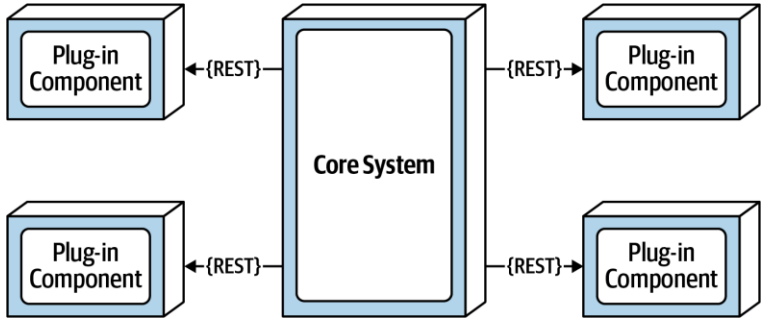
- implemented as shared libraries (such as a JAR, DLL, or Gem) or



- as a separate namespace or package name within the same code base or IDE project



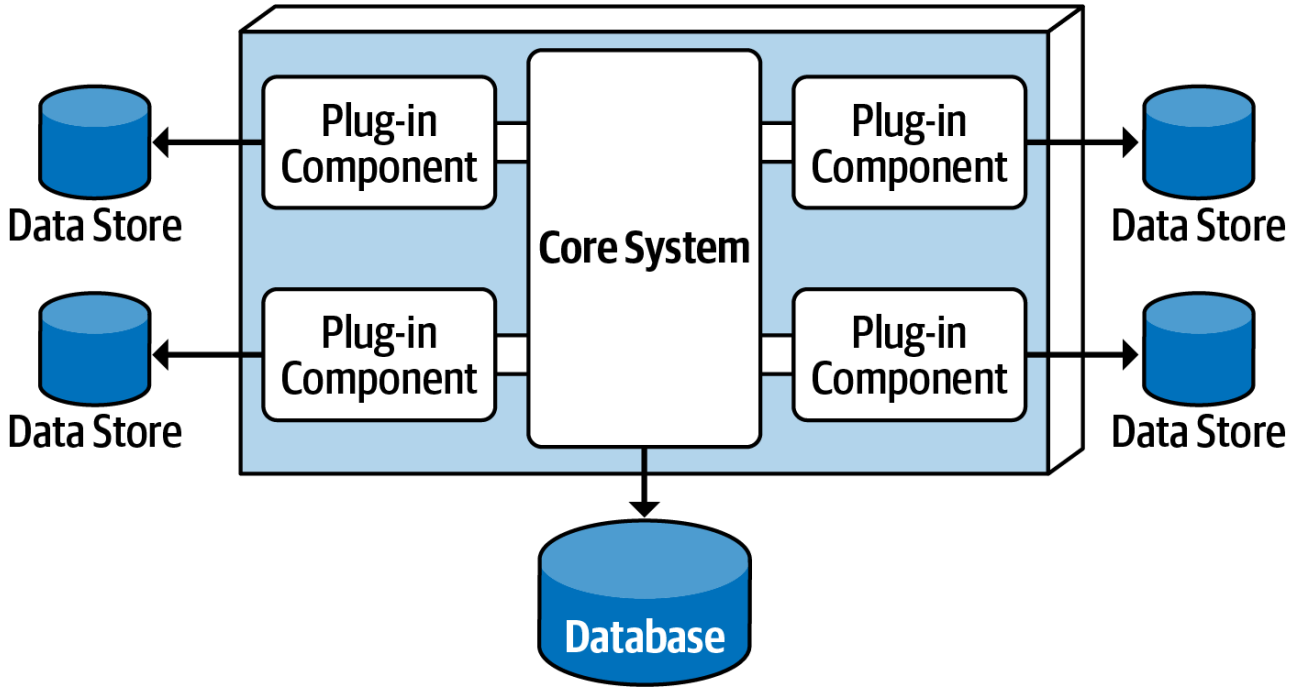
OTHER COMMUNICATION TYPES



- Remote plug-in using REST
- **Benefits**
  - Better decoupling
  - Better scalability and throughput
  - Allows for runtime changes (no need for special frameworks)
  - Allows for asynchronous communication to plug-ins
- **Trade-offs**
  - Microkernel becomes distributed => difficult to deploy for 3rd party on-prem products
  - More complex, additional cost

DATA ACCESS

- Plug-ins do not connect directly to a centrally shared database
- The core system does access the database
- Plug-ins can have their **own separate data stores** only accessible to that plug-in



REGISTRY

- How does the core system know what plug-ins are available?

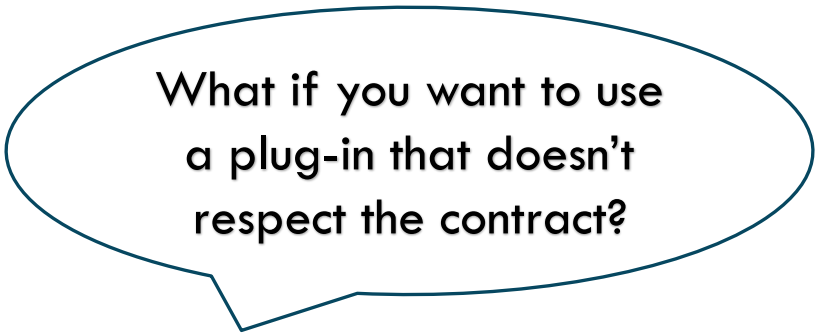
⇒A plug-in **registry** containing:

- Name of plug-in
- Data contract
- Remote access protocol details

- Registry can be implemented as
  - Simple internal map structure (key – plug-in reference) or
  - Complex registry and discovery tool
    - Within core system or
    - Deployed externally (ex. Apache ZooKeeper, Consul)

CONTRACTS

- between the plug-in components and the core system
- usually standard across a domain of plug-in components
- include **behavior, input data,** and **output data** returned from the plug-in component
- can be implemented in XML, JSON, or even objects



EXAMPLES

- Eclipse IDE, PMD, Jira, Jenkins, Chrome, Firefox
- An electronic device recycling application must perform specific custom assessment rules for each electronic device received.

```
public void assessDevice(String deviceID) {
   if (deviceID.equals("iPhone6s")) {
      assessiPhone6s();
   } else if (deviceID.equals("iPad1"))
      assessiPad1();
   } else if (deviceID.equals("Galaxy5"))
      assessGalaxy5();
   } else ...
      ...
   }
}
```

EXAMPLE

- the core system locates and invokes the corresponding device plug-ins

```
public void assessDevice(String deviceID) {
	String plugin = pluginRegistry.get(deviceID);
	Class<?> theClass = Class.forName(plugin);
	Constructor<?> constructor =
theClass.getConstructor();
	DevicePlugin devicePlugin =
		(DevicePlugin)constructor.newInstance();
	DevicePlugin.assess();
}
```

EXAMPLE REGISTRY

- a simple registry within the core system, showing a point-to-point entry, a messaging entry, and a RESTful entry example for assessing an iPhone 6S device

```
Map<String, String> registry = new HashMap<String,
String>();

static {

  //point-to-point access example

  registry.put("iPhone6s", "Iphone6sPlugin");

  //messaging example

  registry.put("iPhone6s", "iphone6s.queue");

  //restful example


registry.put("iPhone6s","https://atlas:443/assess/iphone6s
");

}
```

EXAMPLE CONTRACT

- contract (implemented as a standard Java interface named AssessmentPlugin) defines the overall behavior (assess(), register(), and deregister()), along with the corresponding output data expected from the plug-in component (AssessmentOutput)

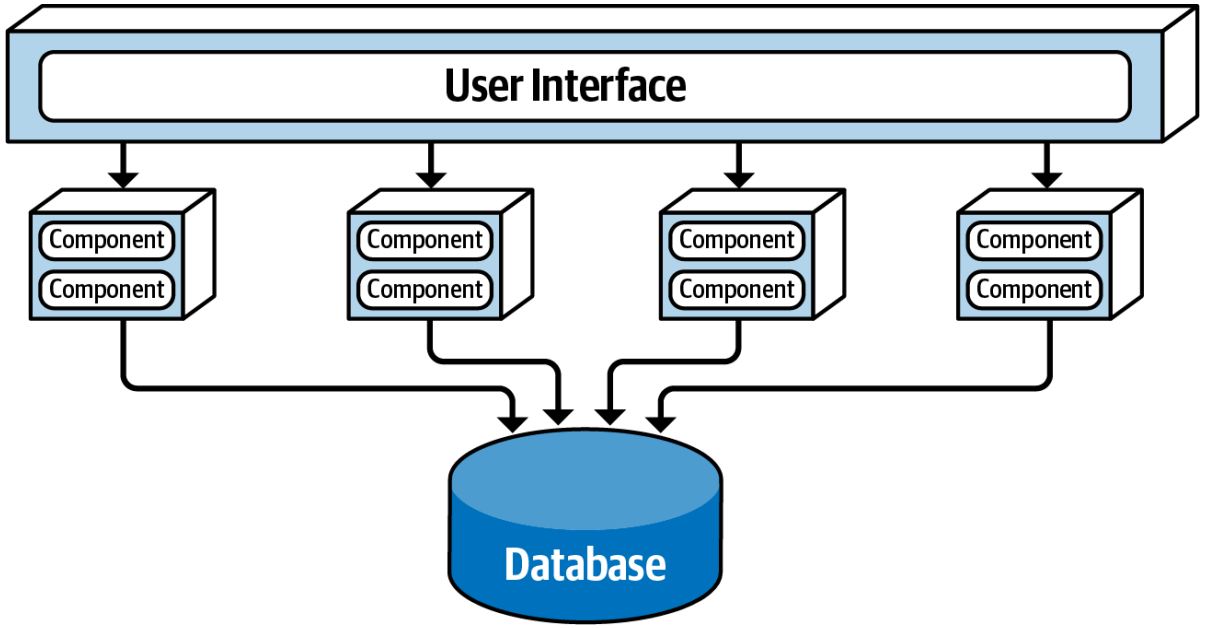
```
public interface AssessmentPlugin {
        public AssessmentOutput assess();
        public String register();
        public String deregister();
}
public class AssessmentOutput {
        public String assessmentReport;
        public Boolean resell;
        public Double value;
        public Double resellPrice;
}
```

ARCHITECTURE CHARACTERISTICS RATINGS

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★★
Reliability	★★★
Scalability	★
Simplicity	★★★★
Testability	★★★

SERVICE-BASED ARCHITECTURE STYLE

- Topology: **distributed** macro layered structure
  - Separately deployed user interface
  - Separately deployed remote coarse-grained services
  - Monolithic database

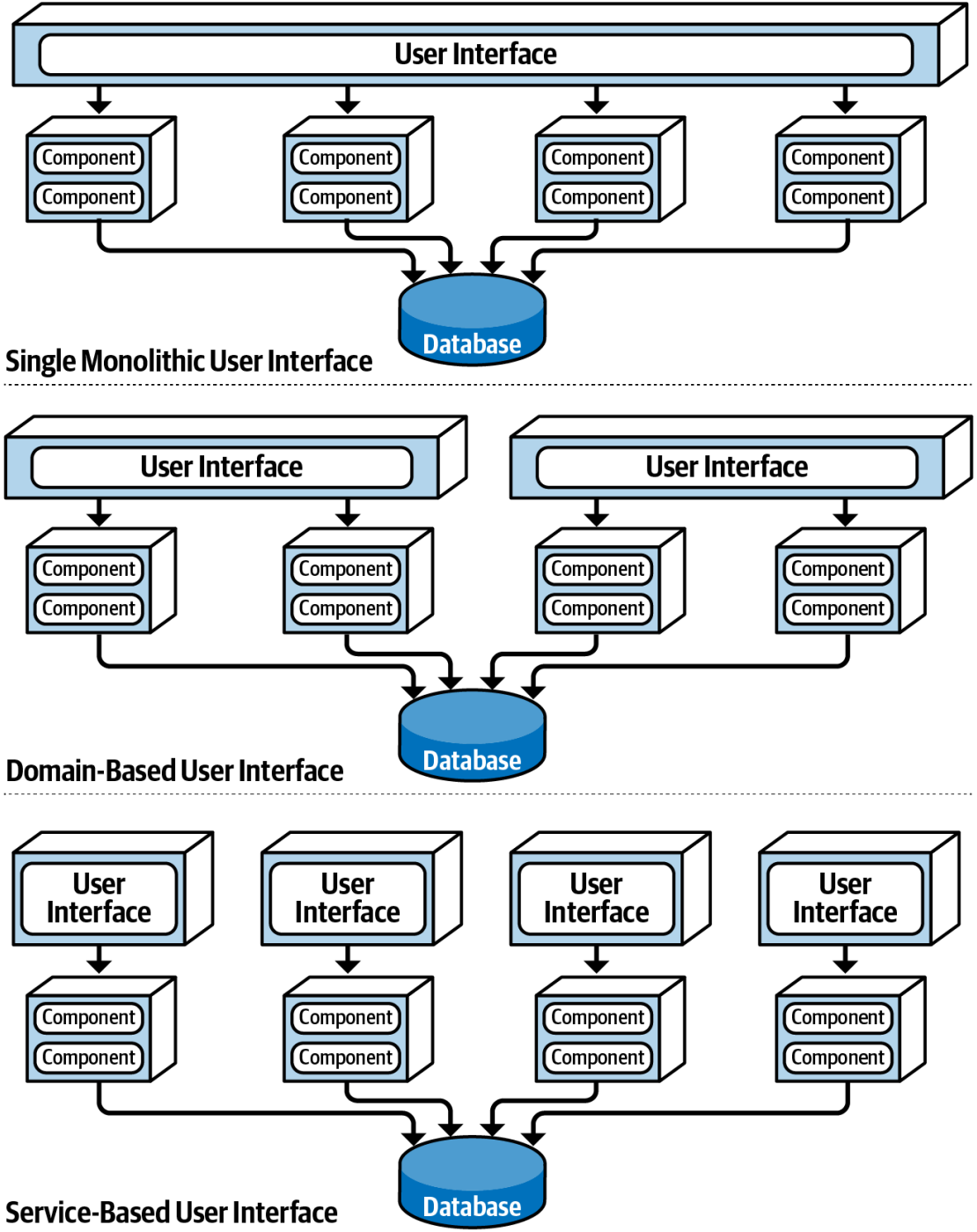


SERVICES

- Coarse grained “domain” services
- Independent
- Separately deployed as EAR file, WAR file or assembly => no containerization needed
- Usually, one single instance of each domain service
- If multiple instances of a service => load balancing mechanisms needed
- Accessed remotely from a user interface using a remote access protocol (i.e. REST, messaging, RPC, SOAP)

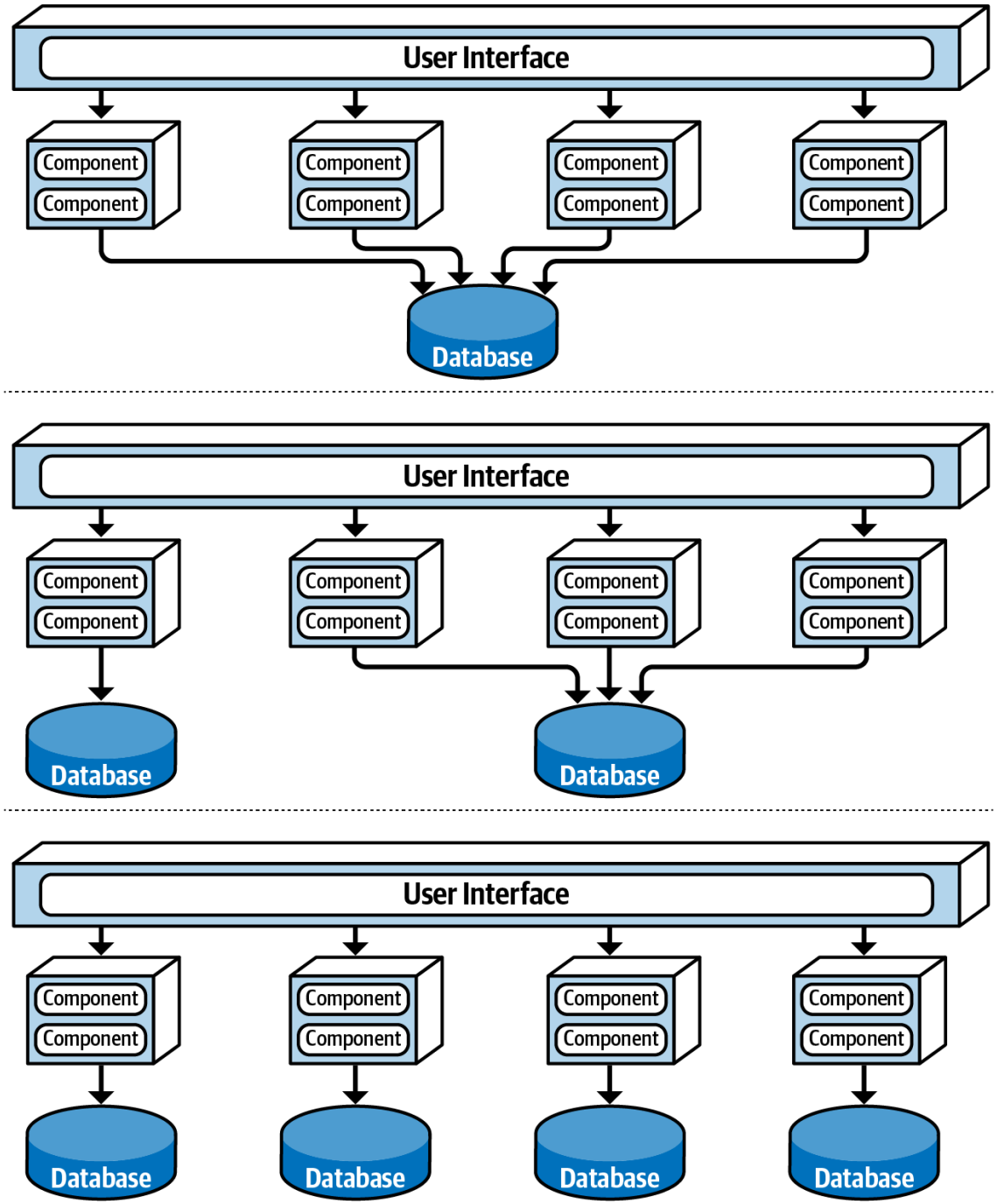
TOPOLOGY VARIANTS

- User Interface variants



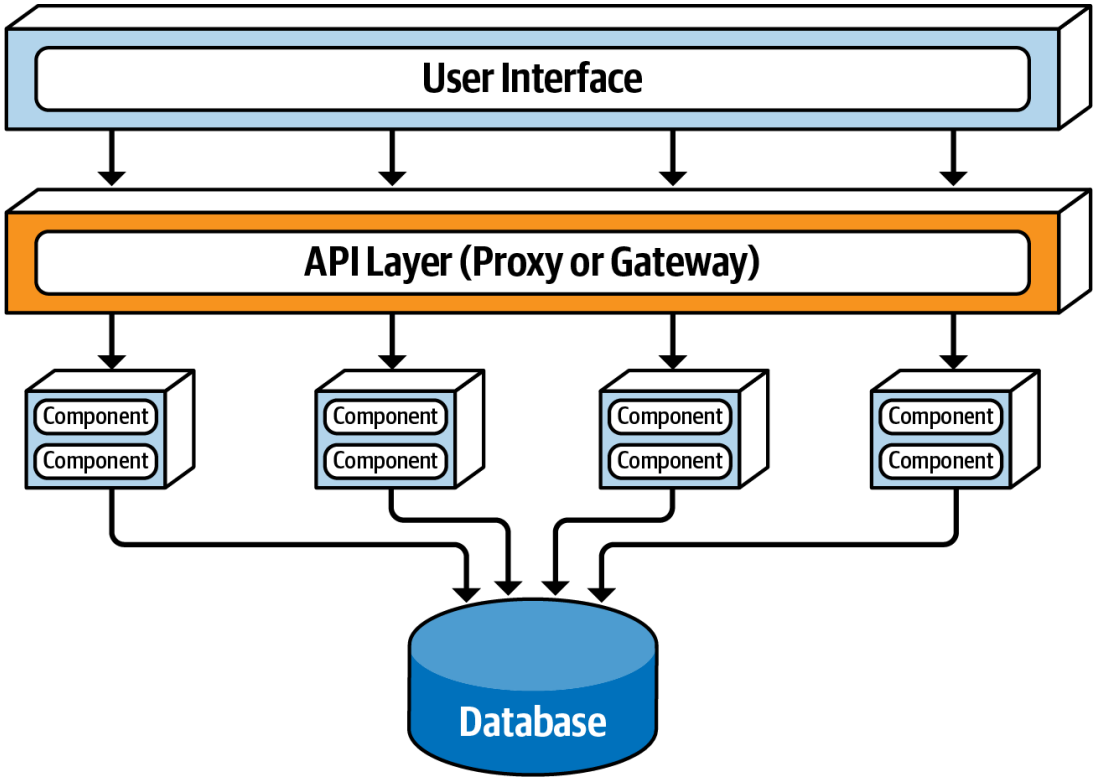
TOPOLOGY VARIANTS

- Database variants
- make sure the data in each separate database is not needed by another domain service!



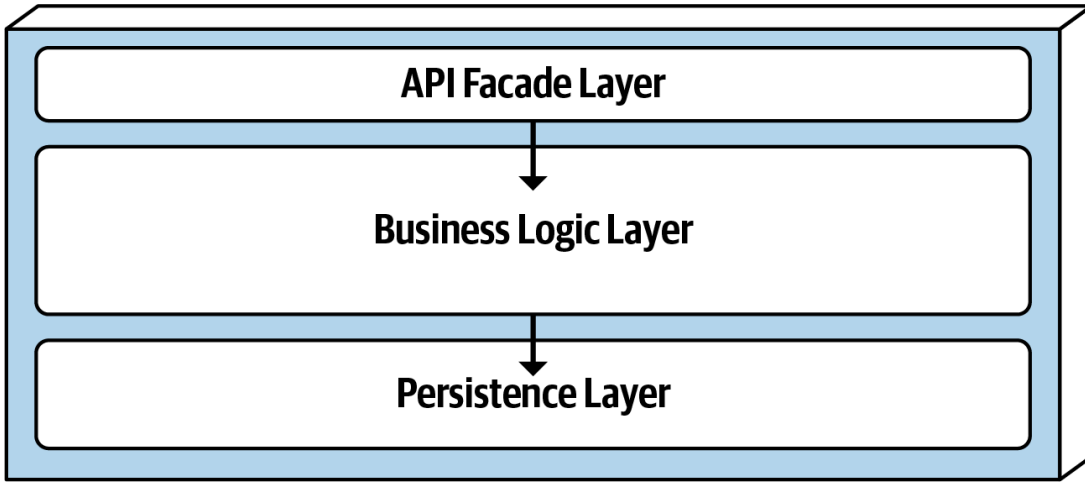
TOPOLOGY VARIANTS

- Adding an API layer
- Good for exposing service functionality to external systems



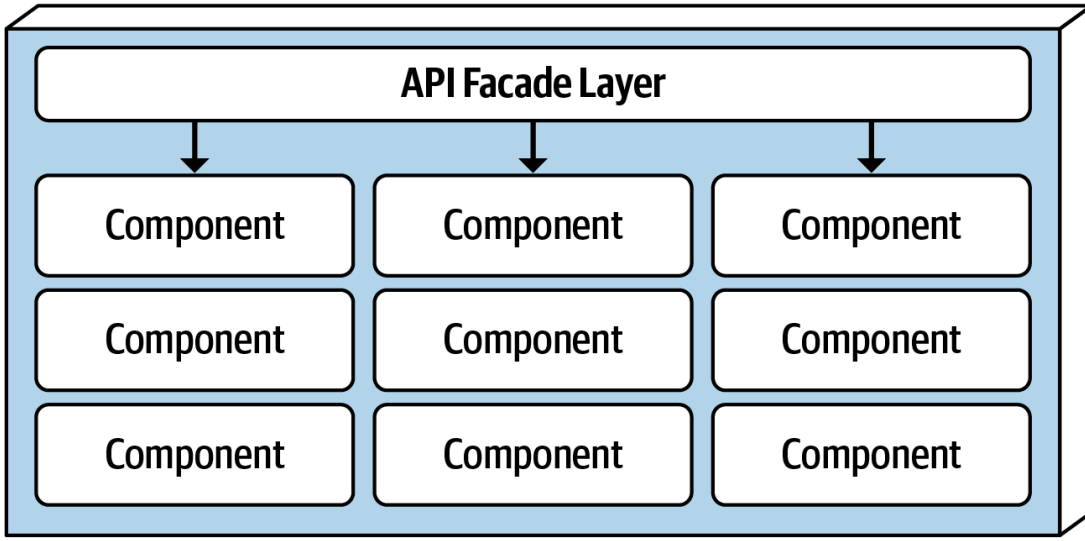
SERVICE DESIGN

- Must contain an API access façade
- API has also orchestration responsibilities (internal class-level orchestration)
- Regular ACID database transactions apply within a single domain service
- More difficult to test



Layered design (technical partitioning)

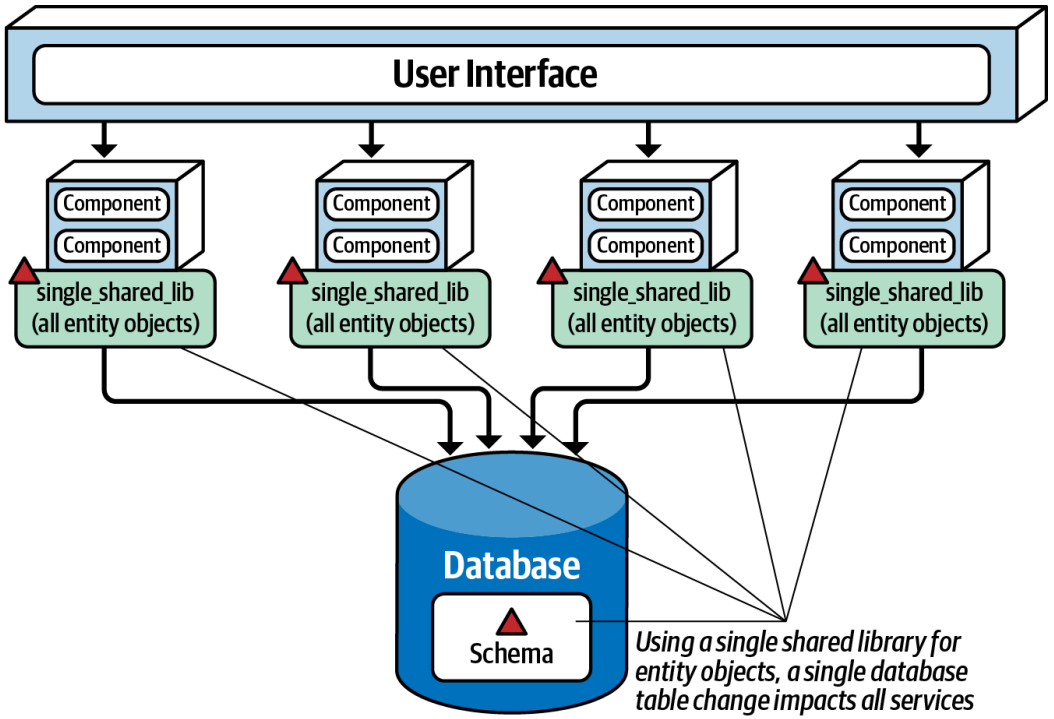
Domain design (domain partitioning)



DATABASE PARTITIONING

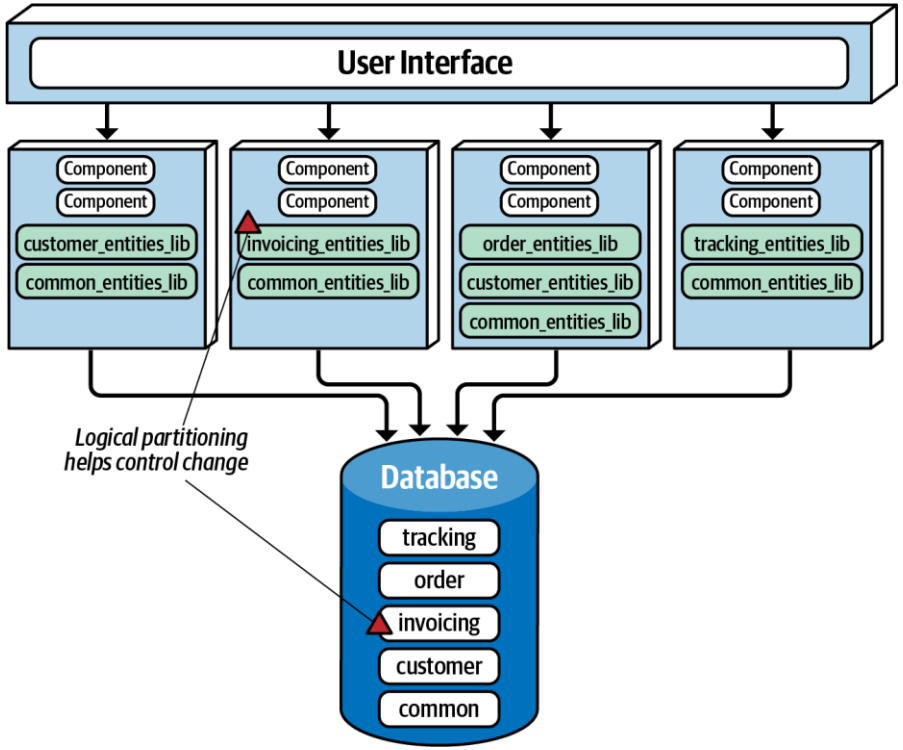
- Strong coupling to the database
- Worst practice = creating a single shared library of entity objects

=> partition!



DATABASE PARTITIONING

- Logical partitions => corresponding shared libraries
- Make the logical partitioning in the database as fine-grained as possible while still maintaining well-defined data domains

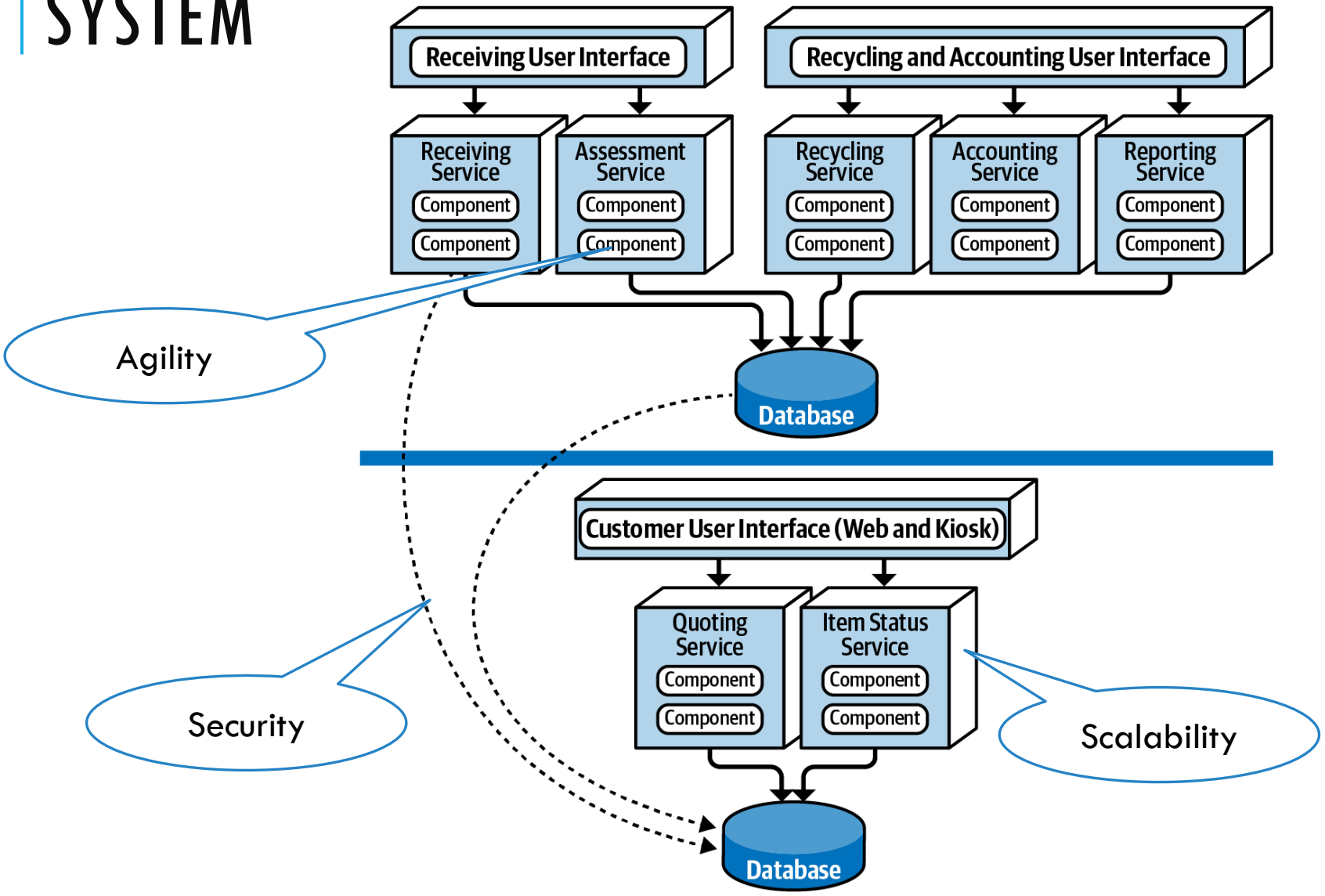


EXAMPLE ELECTRONIC RECYCLING SYSTEM

Processing flow:

1. The customer asks the company (via a website or kiosk) how much money they can get for the old electronic device (**quoting**)
2. If ok, the customer will send the electronic device to the recycling company, which will receive the physical device (**receiving**)
3. The recycling company will then assess the device to determine if the device is in good working condition or not (**assessment**)
4. If the device is in good working condition, the company will send the customer the money promised for the device (**accounting**)
5. Through this process, the customer can go to the website at any time to check on the status of the item (**item status**)
6. Based on the assessment, the device is then recycled by either safely destroying it or reselling it (**recycling**)
7. the company periodically runs ad hoc and scheduled financial and operational reports based on recycling activity (**reporting**).

EXAMPLE ELECTRONIC RECYCLING SYSTEM



ARCHITECTURE CHARACTERISTICS RATINGS

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★

ARCHITECTURE CHARACTERISTICS RATINGS

- a **domain-partitioned** architecture

Benefits

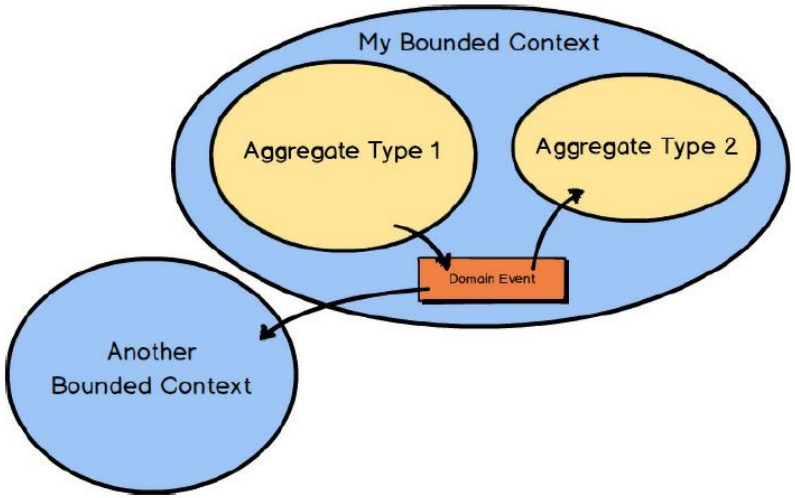
- Agility (faster change)
- Testability (better test coverage due to the limited scope of the domain)
- Deployability (ability for more frequent deployments carrying less risk than a large monolith)
- Services are usually self-contained and do not leverage interservice communication due to database sharing => fault tolerance
- Larger services => less network traffic to and between services, fewer distributed transactions, and less bandwidth used => increased overall reliability with respect to the network

WHEN TO USE IT

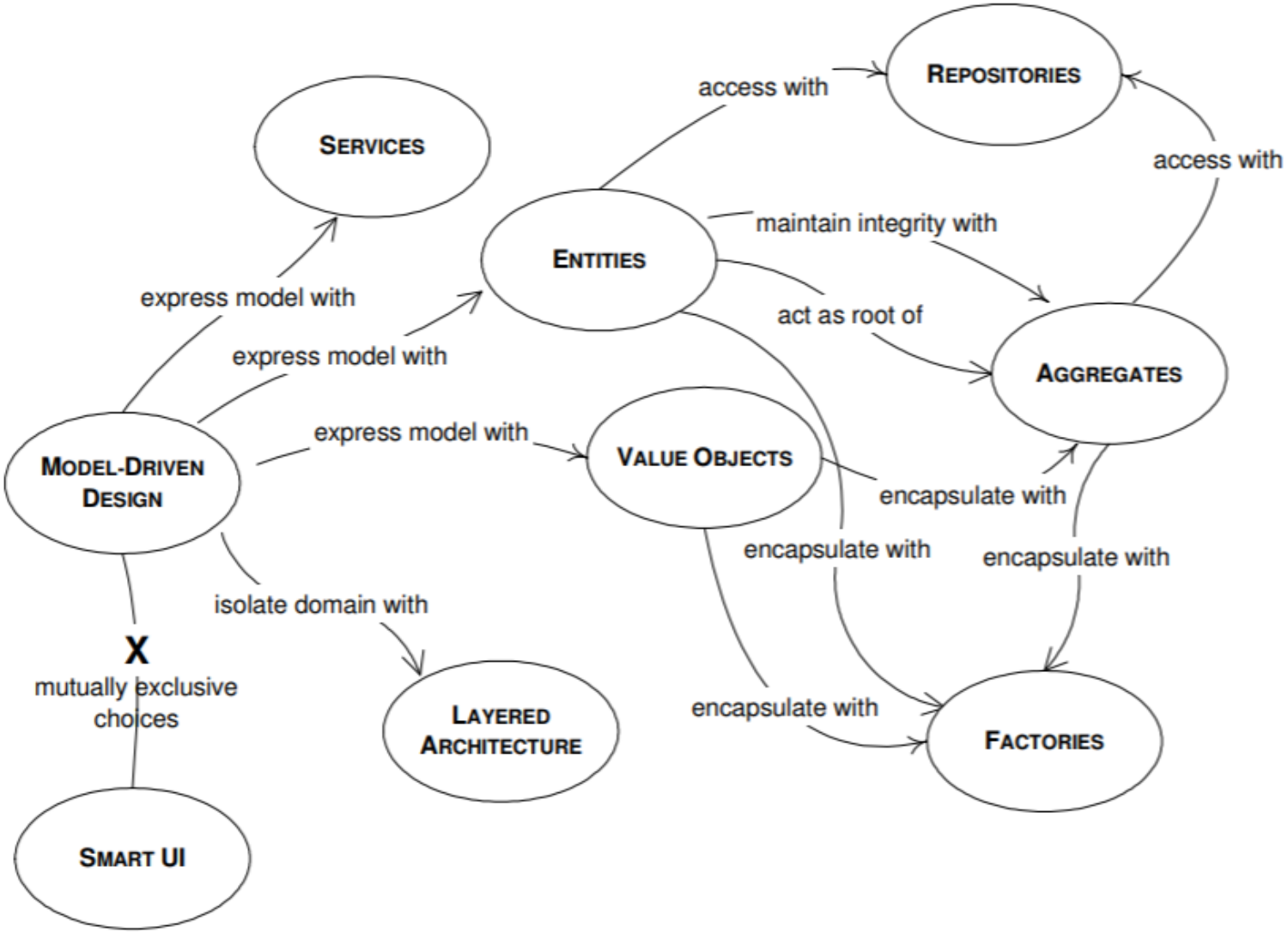
- A natural fit when doing domain-driven design
- Each service in service-based architecture encompasses a particular domain
- In most cases the transaction is scoped to a particular domain service => ACID transactions
- Good trade-off between modularity and complexity (no complex orchestration/choreography is needed)

DOMAIN DRIVEN DESIGN

- **Strategic** Design
  - Subdomains (**problem** perspective)
    - Core Domain
    - Support Domain
    - Generic Domain
  - Bounded Context (**solution** perspective)
  - Context Mapping
- **Tactical** Design
  - Aggregates
  - Domain Events



MODEL DRIVEN DESIGN

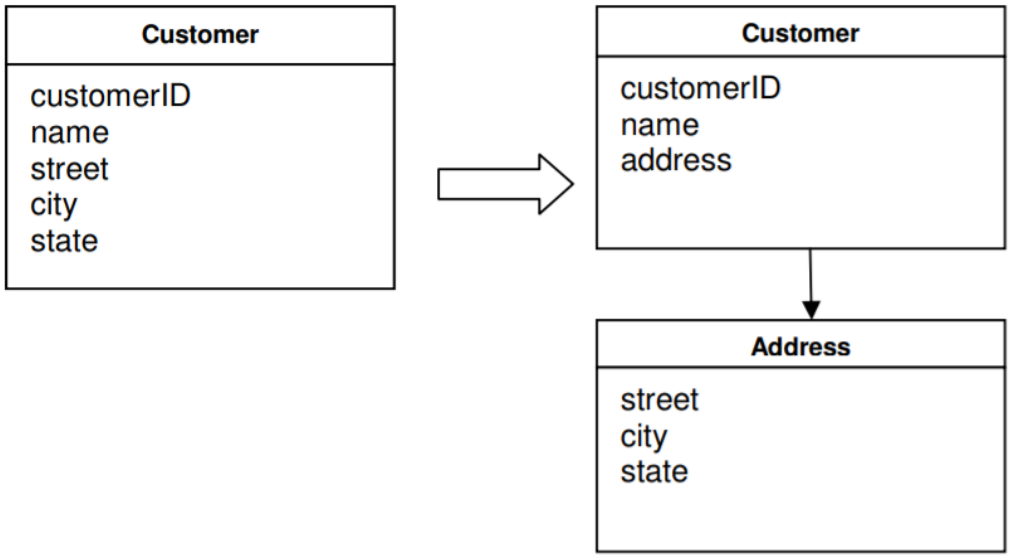


ENTITIES

- Have an **identity** that remains unchanged through the states of the application
- Focus on identity and continuity, not on values of the attributes
- The identity can be
  - An attribute
  - A combination of attributes
  - An artificial attribute
- Examples:
  - Person
  - Account

VALUE OBJECTS

- Do **not have identities**
- The **values** of the attributes are important
- Easily created and discarded (no tracking is needed)
- Highly recommended to make them immutable => can be shared!



ENTITY OR VALUE OBJECT?

- **Entity**
  - Need to decide how to uniquely identify it
  - Need to track it
  - Need one instance for each object => affects performance

  Ex. Customer object

- **Value Object**
  - If the Value Object is shareable, it should be immutable
  - Can contain other Value Objects and even references to Entities

SERVICES

- Contain operations that do not belong to any Entity/Value object
- Do not have internal states
- Operate on Entity/Value objects becoming a point of connection => loose coupling between objects

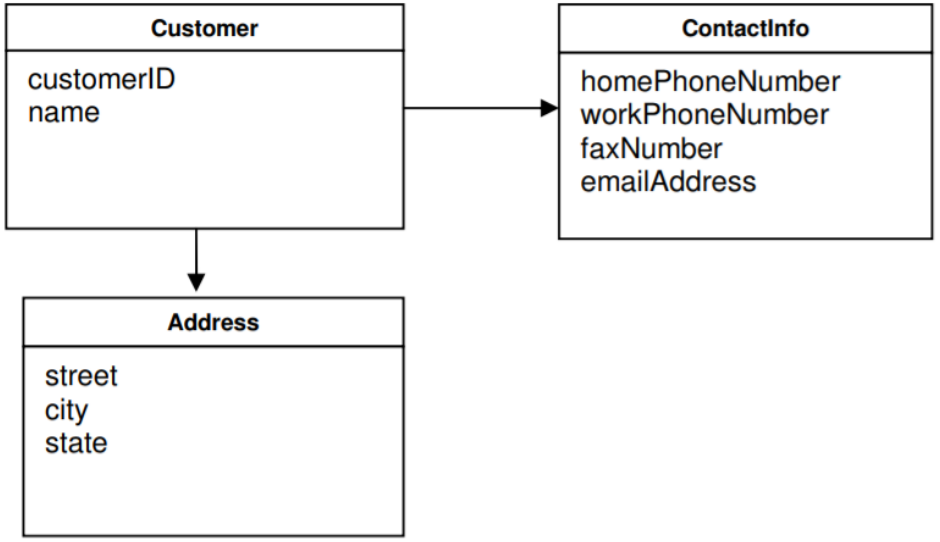
SERVICES

Characteristics:

- The operation performed by the Service refers to a domain concept which does not naturally belong to an Entity or Value Object.
- The operation performed refers to other objects in the domain.
- The operation is stateless.

Ex. transferring money from one account to another.

AGGREGATES

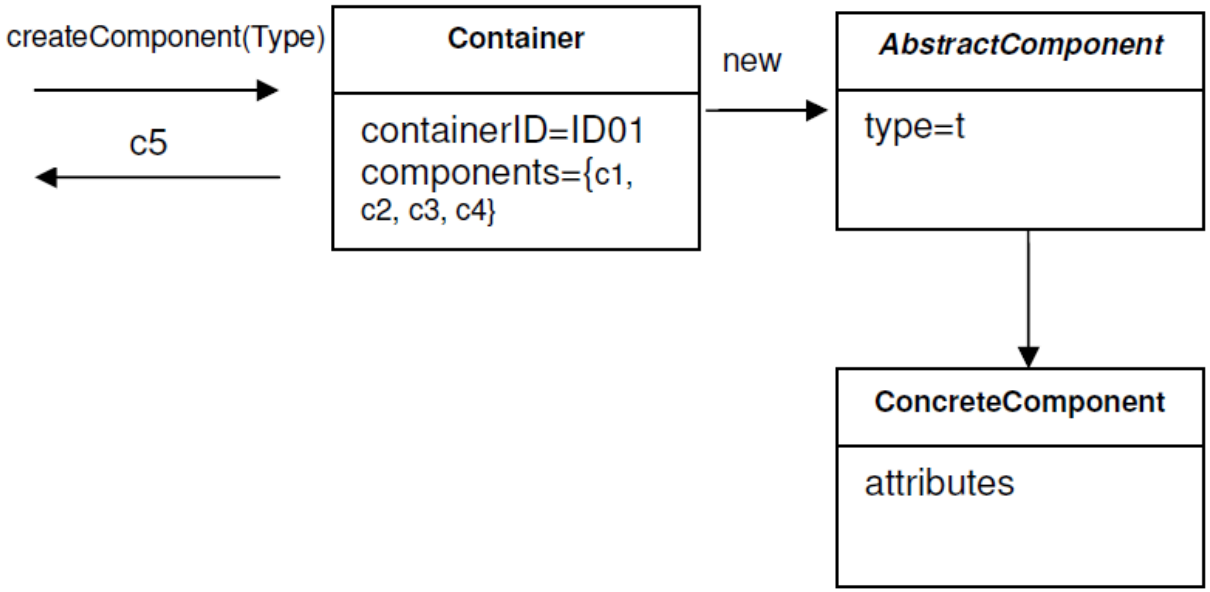


- **A domain pattern used to define object ownership and boundaries**
- Associations
  - One-to-one
  - One-to-many
  - Many-to-many
- Groups associated objects that represent one unit with regard to data change => defines a **transactional consistency boundary**
- Each aggregate has one root (an Entity)
- Only the root is accessible from outside

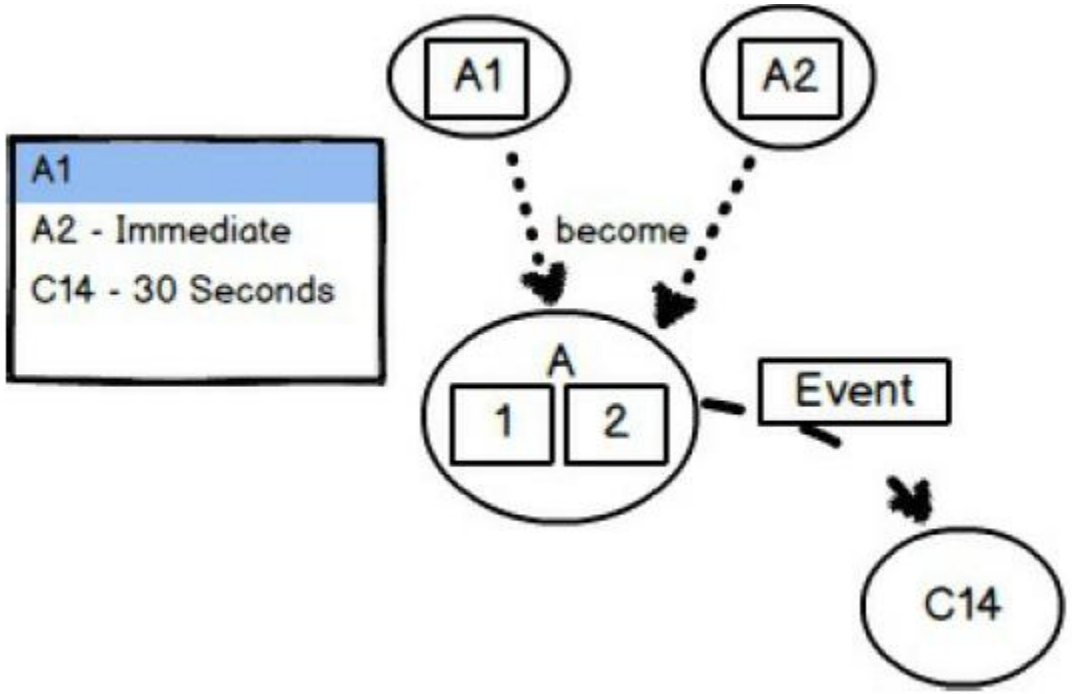
DISCUSSION

- Root Entity has **global** identity, inner entities have **local** identity
- Root Entity enforces invariants
- If the root is deleted => all aggregated objects are deleted, too
- Creating aggregates => atomic process

⇒Use Factories



AGGREGATES



Rules of thumb

- Protect business invariants inside *Aggregate* boundaries. (transactional consistency within Aggregate)
- Design small *Aggregates*.
- Reference other *Aggregates* by identity only.
- Update other *Aggregates* using eventual consistency. (transactional consistency across Aggregates)

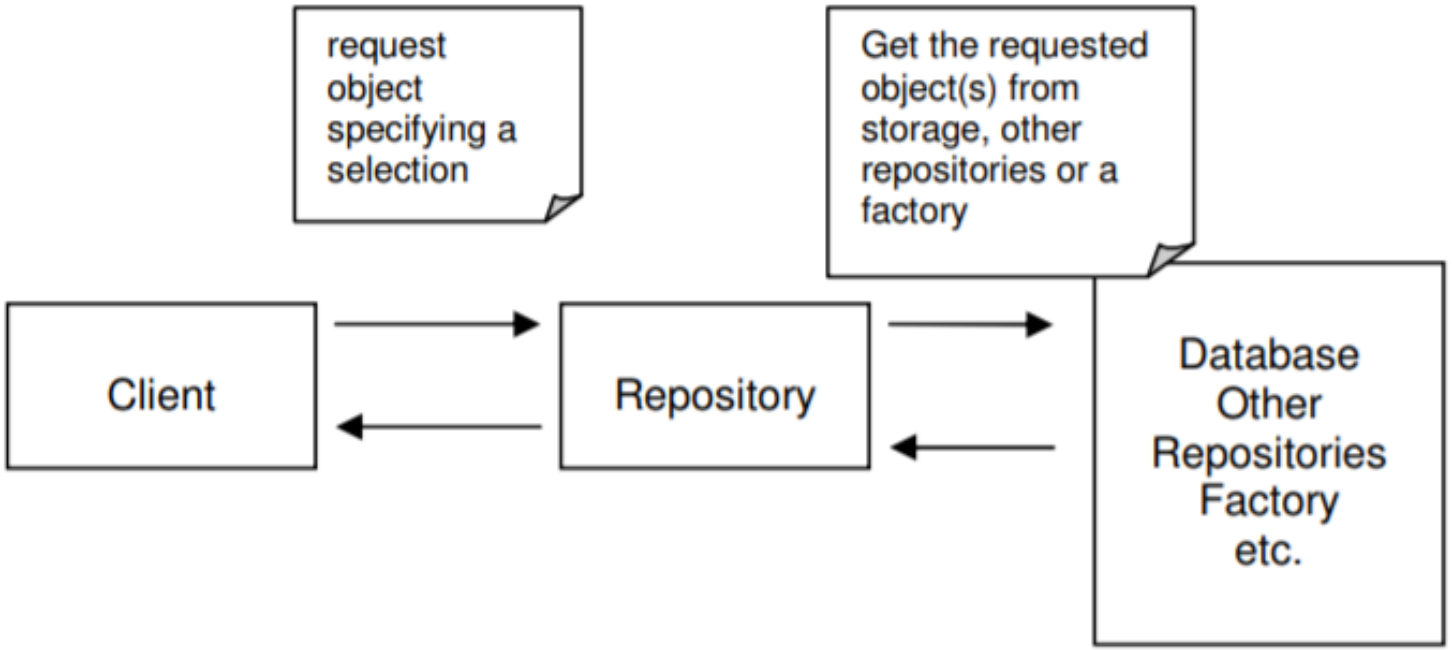
REPOSITORY

- How do we get the object (i.e. Entity, Value)?
  - Create it (Factories)
  - Obtain it
    - If it is a Value Object => need the root of the Aggregate
    - If it is an Entity => can be obtained directly from the database.
  - Problems:
    - Dependency on the database structure
    - Mixing database access into the domain logic

REPOSITORY

Repository encapsulates all the logic needed to obtain object references

- by already storing them OR
- by getting them from the storage).

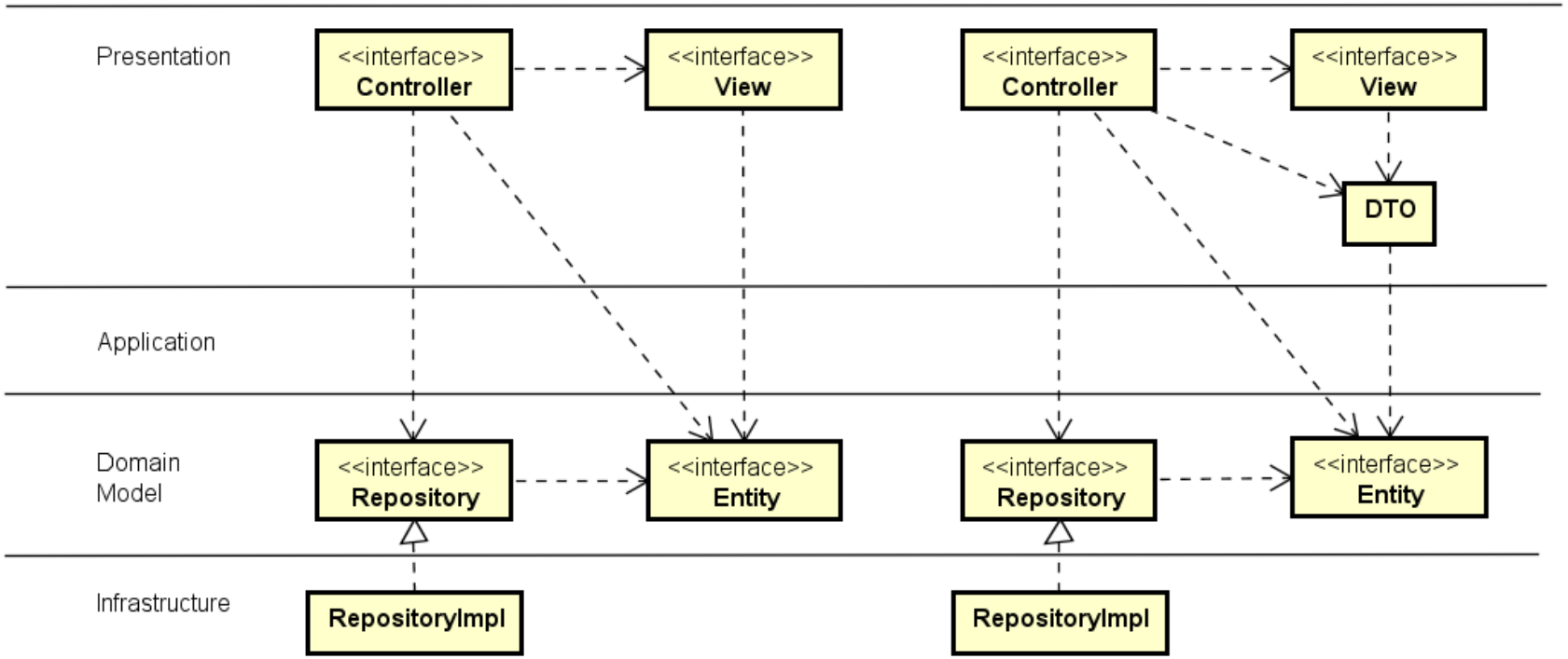


DISCUSSION

How complex should the Domain Model be?

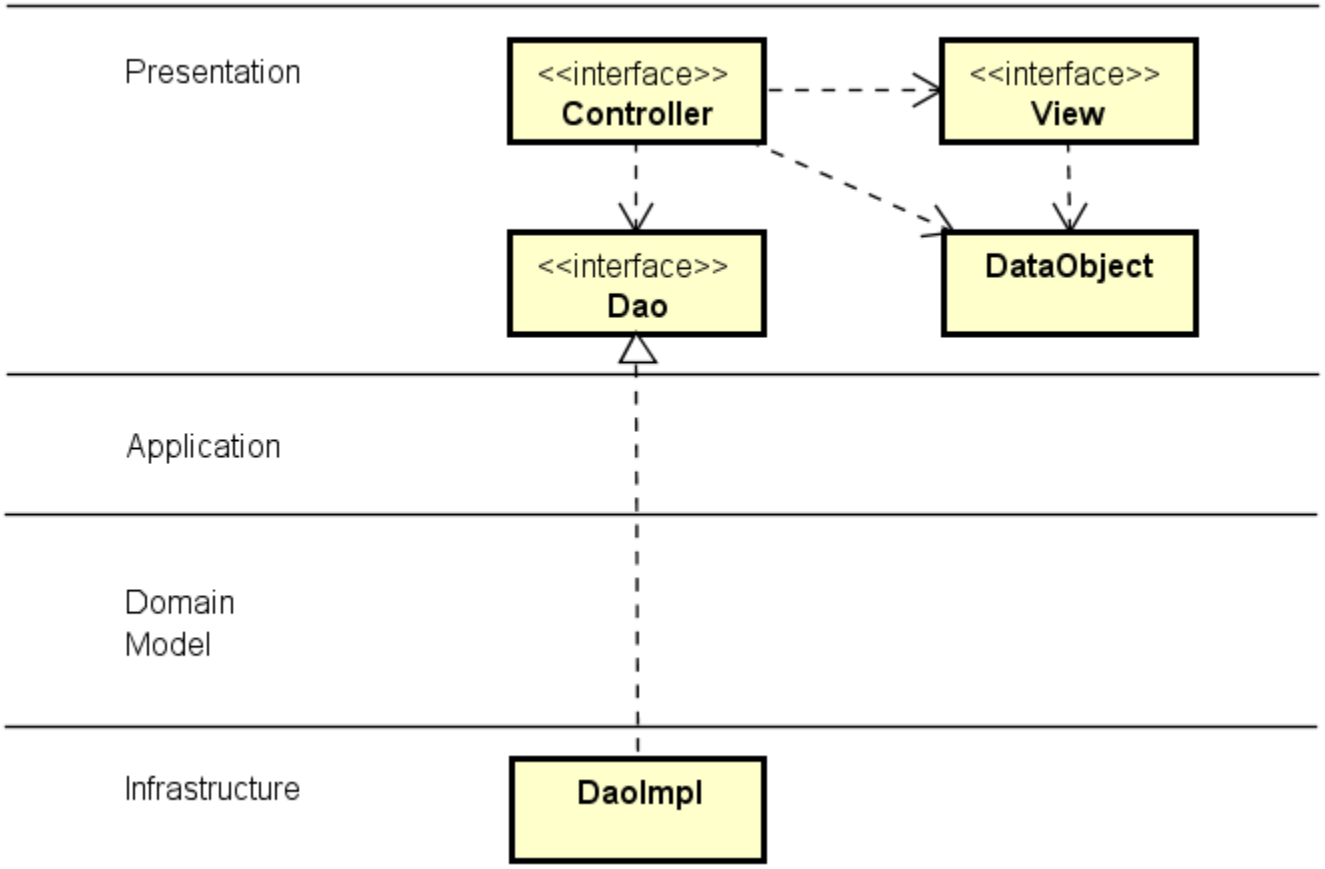
Anemic Domain Model (**Anti-pattern**)

- Just forms over data (CRUD operation + some validation



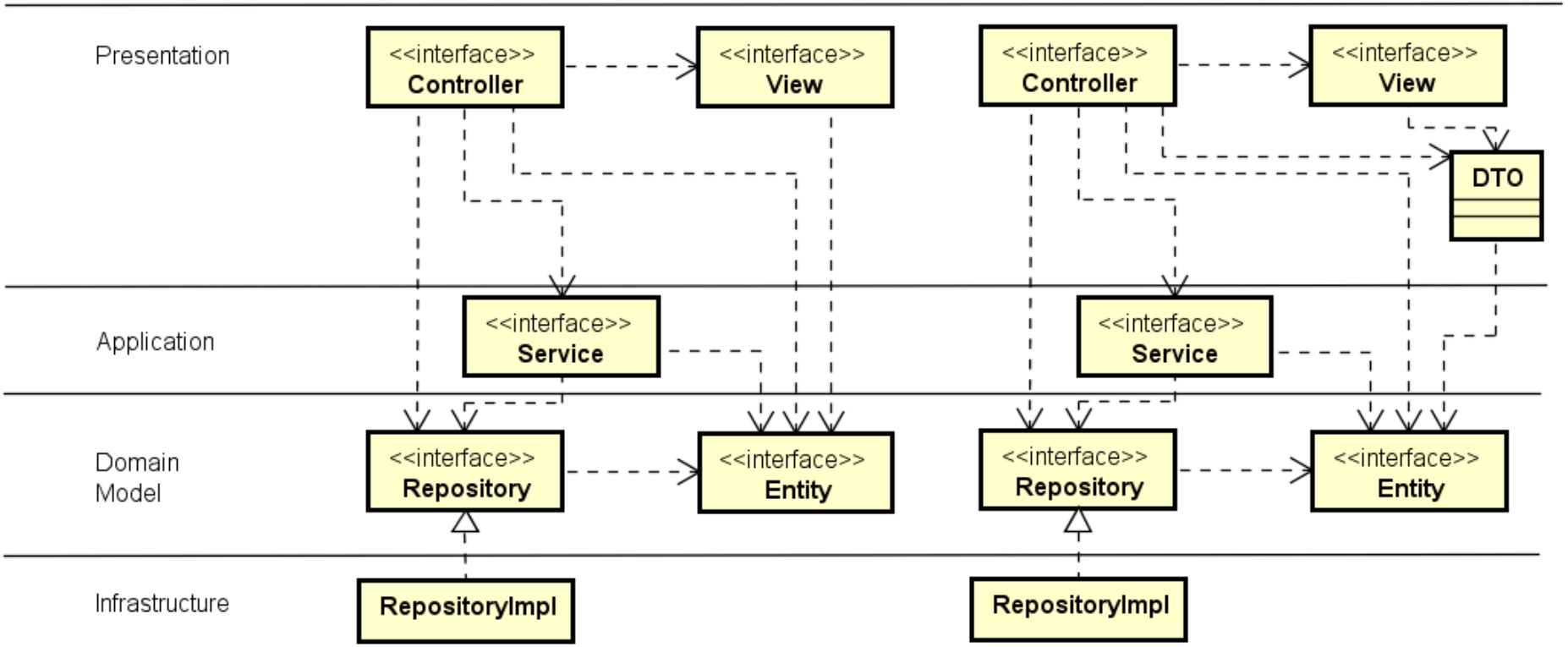
ALTERNATIVES

Even less Domain Model



WHEN TO USE SERVICES?

- Need for driving the workflow, coordination transaction management
- E.g. The controller needs to update more than one entity



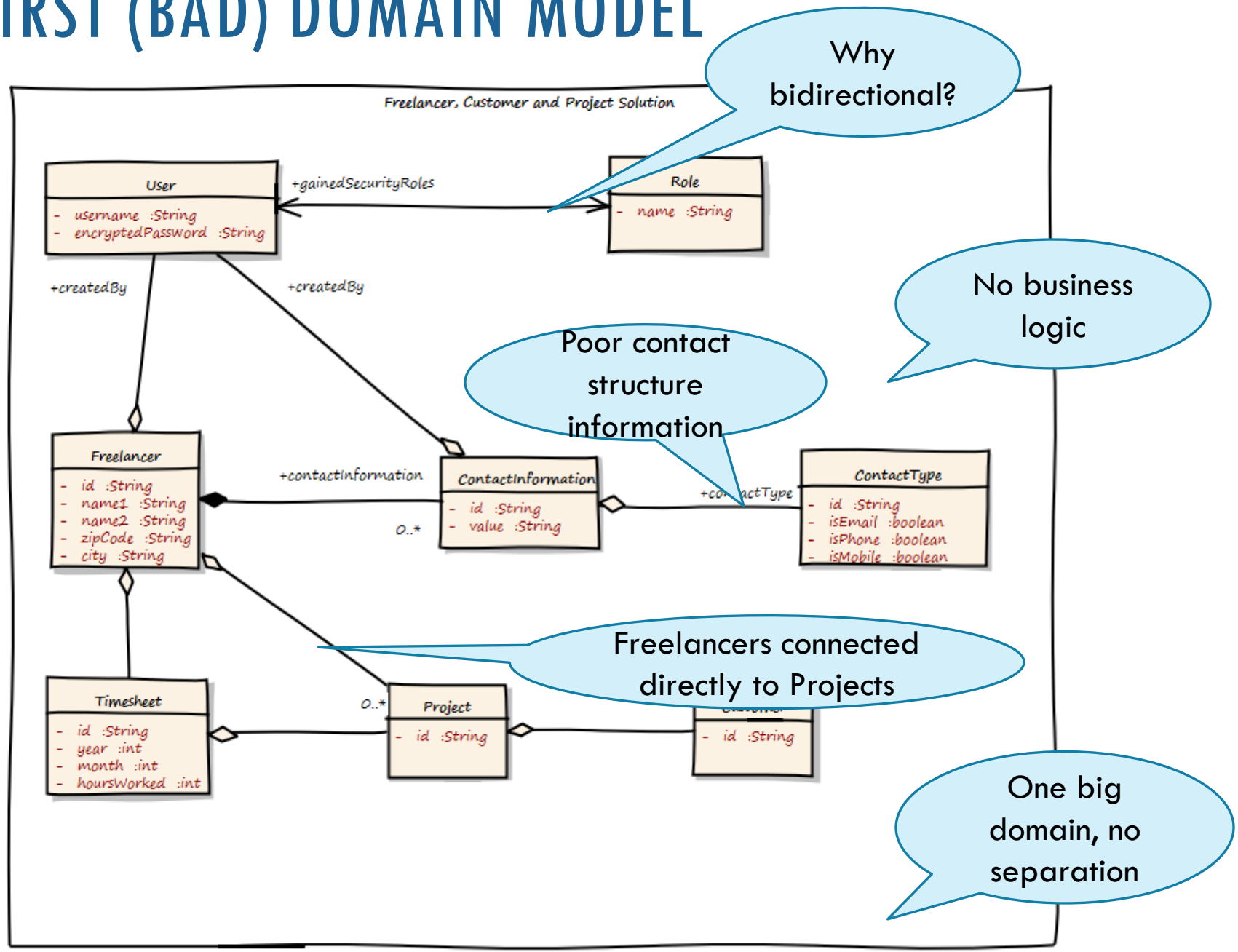
THE IT BODY LEASING EXAMPLE

A company provides IT Body Leasing. They have some Employees, and also a lot of Freelancers as Subcontractors. Requirements:

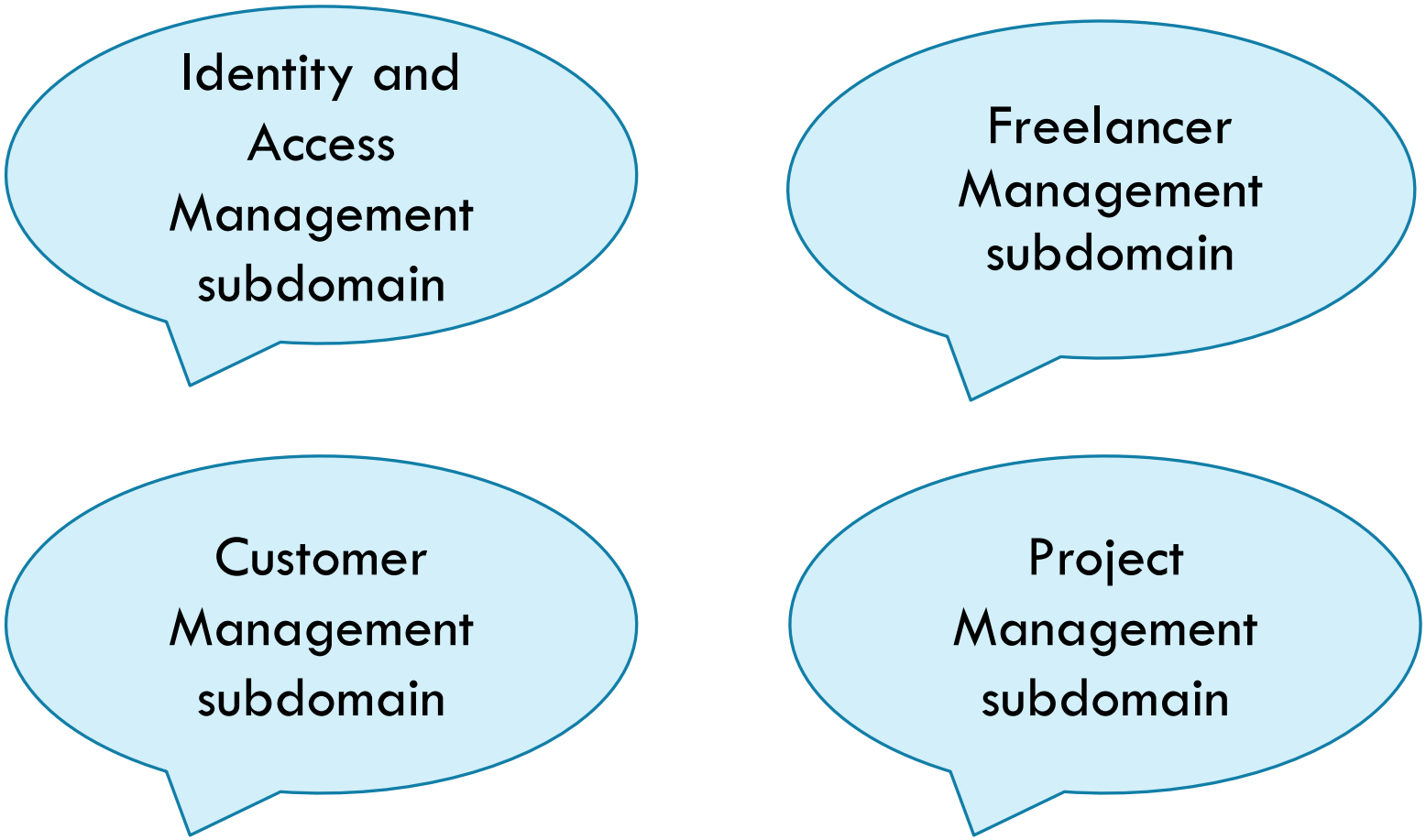
- A searchable catalog of Freelancers must be provided
- Allows to store the different Communication Channels available to contact a Freelancer
- A searchable catalog of Projects must be provided
- A searchable catalog of Customers must be provided
- The Timesheets for the Freelancers under contract must be maintained

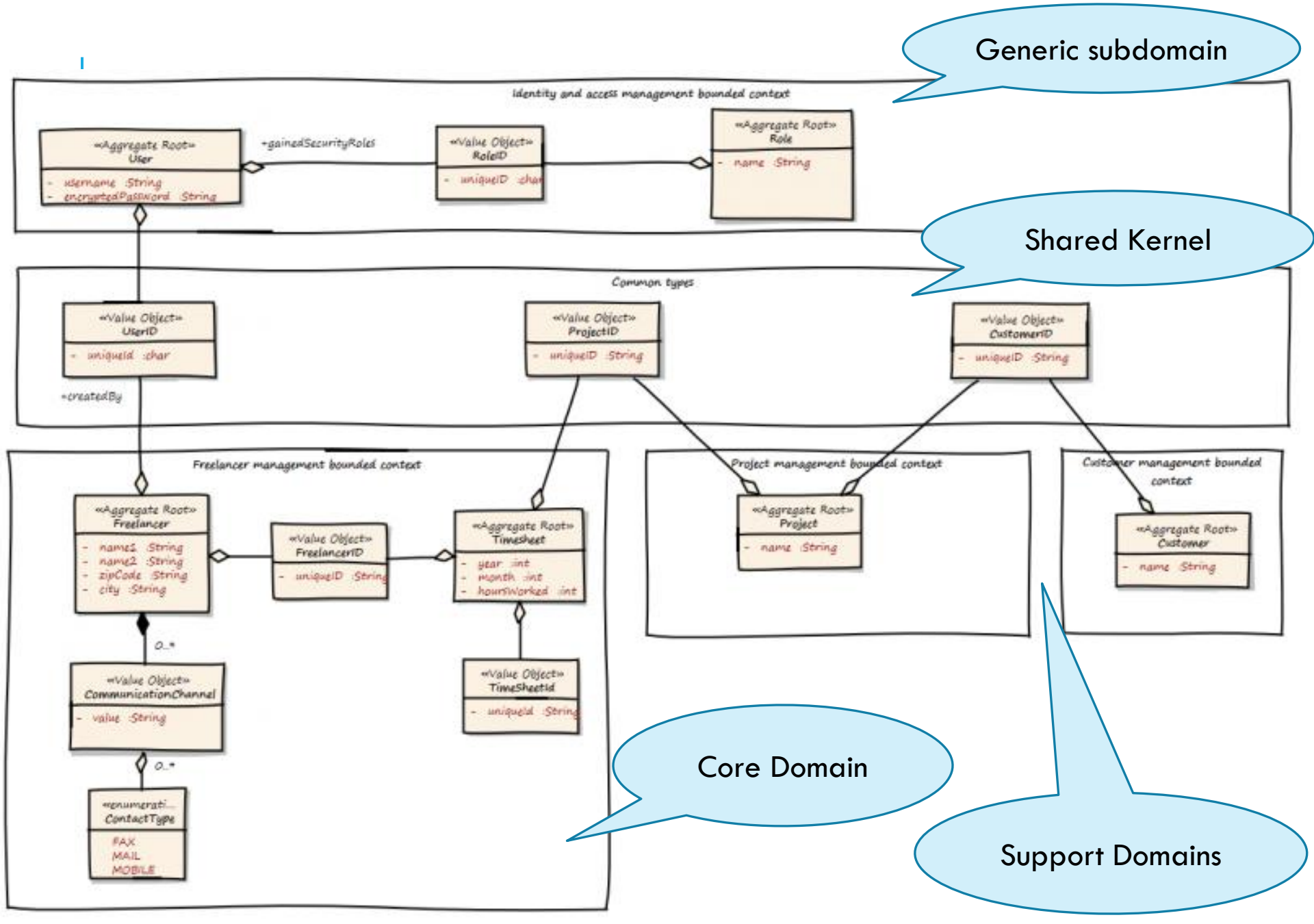
https://www.mirkosertic.de/blog/2013/04/domain-driven-design-example/

FIRST (BAD) DOMAIN MODEL

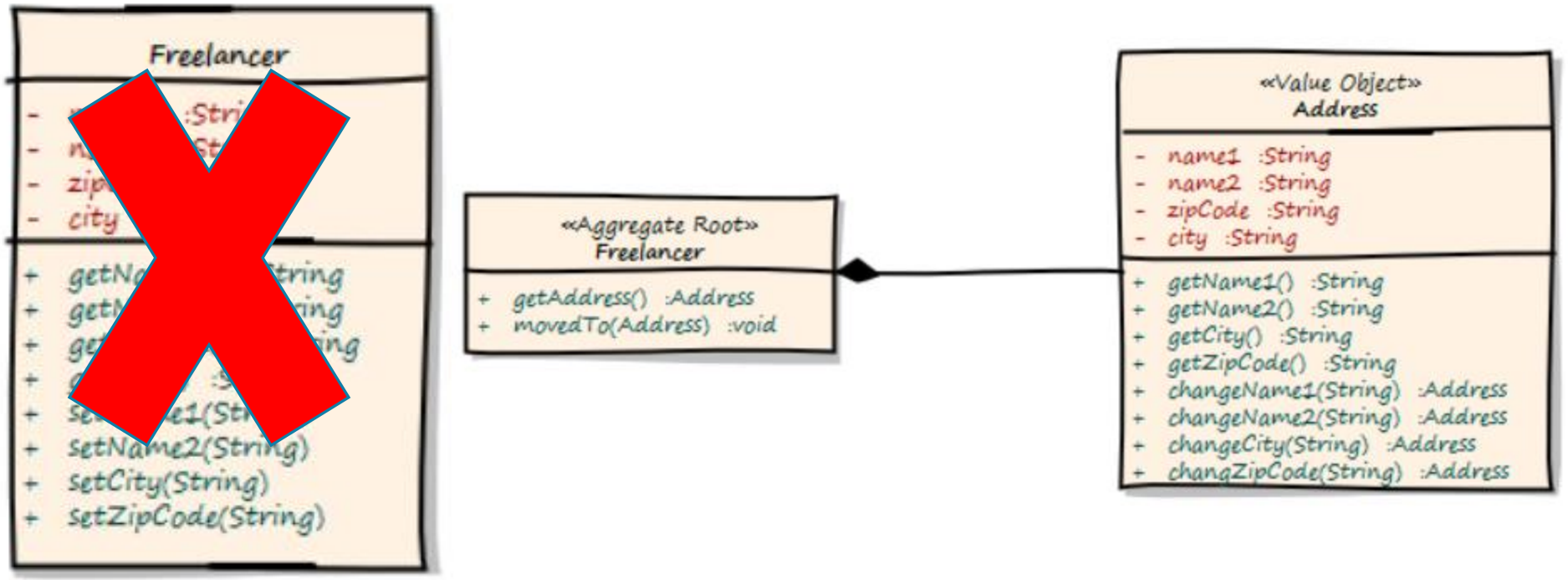


SPLITTING THE PROBLEM INTO SUBDOMAINS



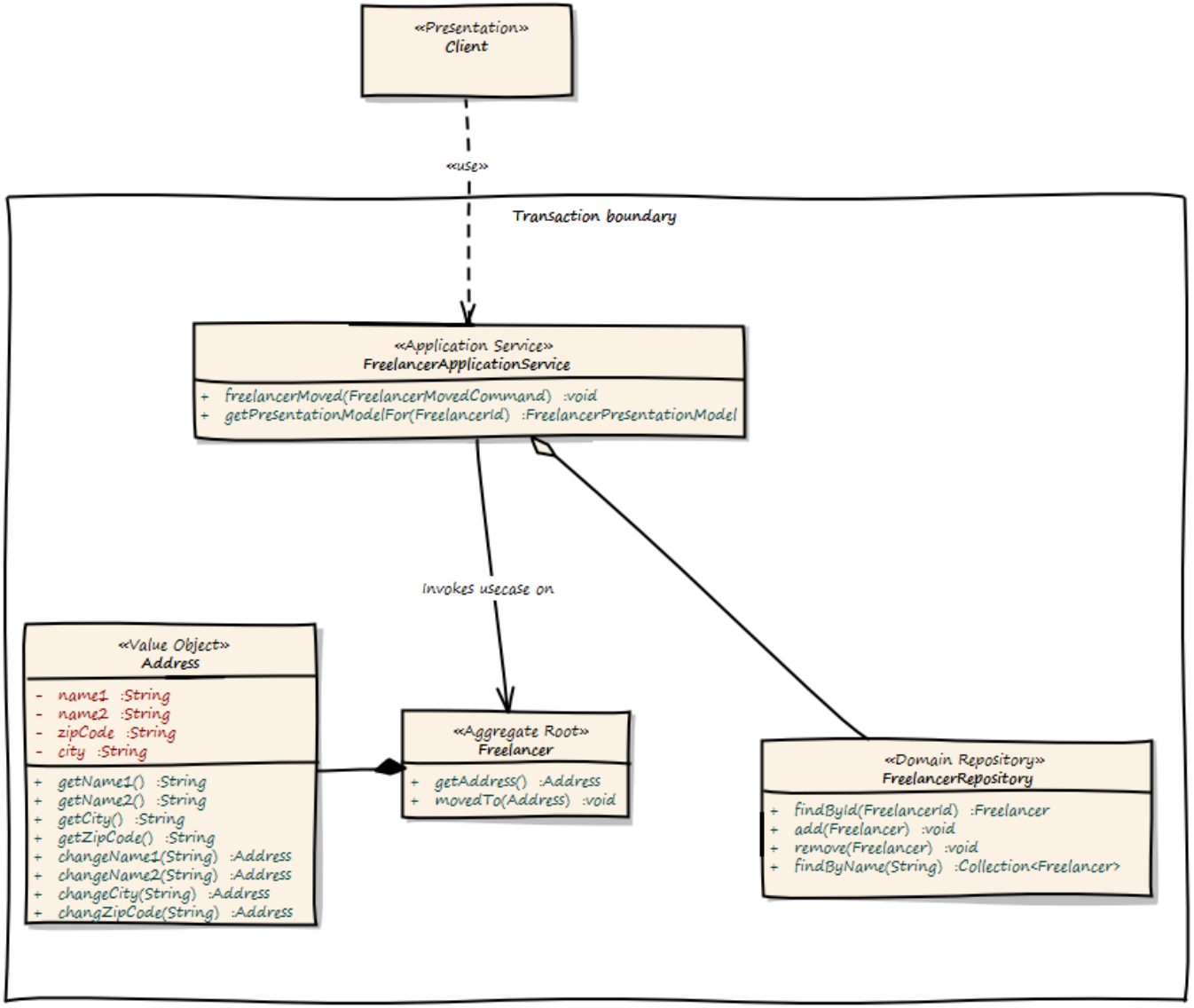

Generic subdomain
Identity and access management bounded context
«Aggregate Root»
User
- username :String
- encryptedPassword :String
+gainedSecurityRoles
«Value Object»
RoleID
- uniqueID :char
«Aggregate Root»
Role
- name :String
Shared Kernel
Common types
«Value Object»
UserID
- uniqueId :char
+createdBy
«Value Object»
ProjectID
- uniqueID :String
«Value Object»
CustomerID
- uniqueID :String
Freelancer management bounded context
«Aggregate Root»
Freelancer
- name1 :String
- name2 :String
- zipCode :String
- city :String
«Value Object»
FreelancerID
- uniqueID :String
«Aggregate Root»
Timesheet
- year :int
- month :int
- hoursWorked :int
0..*
«Value Object»
CommunicationChannel
- value :String
«Value Object»
TimeSheetId
- uniqueId :String
0..*
«enumerati...
ContactType
FAX
MAIL
MOBILE
Project management bounded context
«Aggregate Root»
Project
- name :String
Customer management bounded context
«Aggregate Root»
Customer
- name :String
Core Domain
Support Domains

APPLYING DOMAIN DRIVEN DESIGN

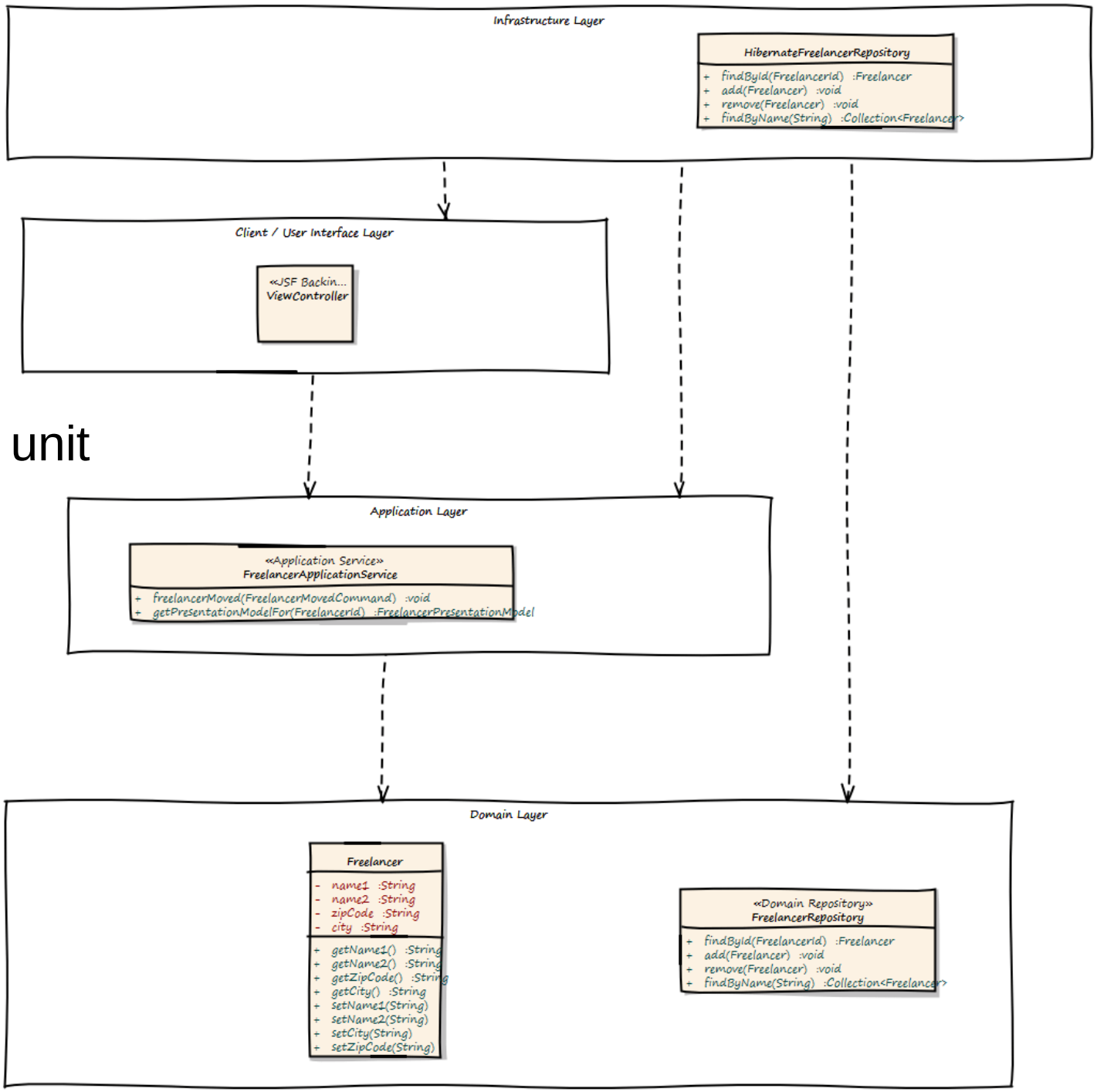


ADDING BEHAVIOR

Freelancer moved to new location



The Deployment unit

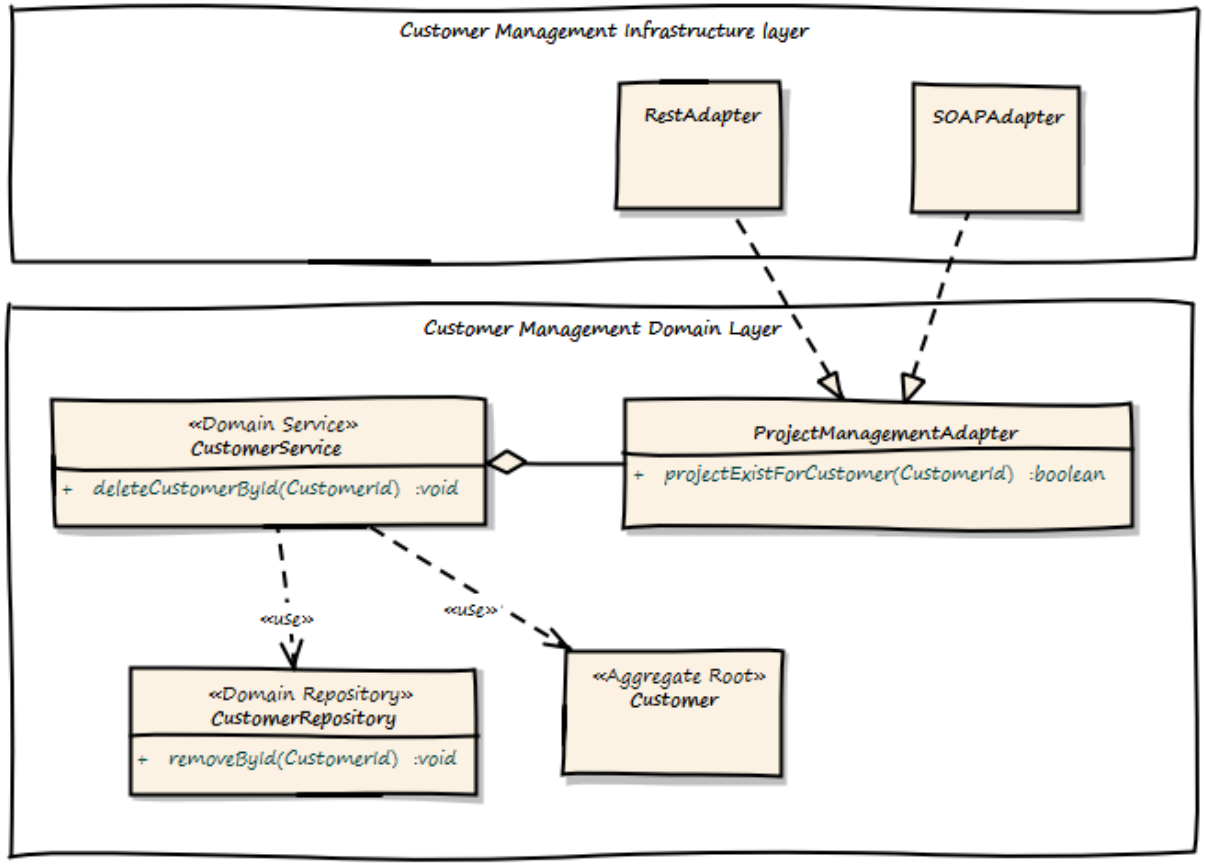


CONTEXT INTEGRATION

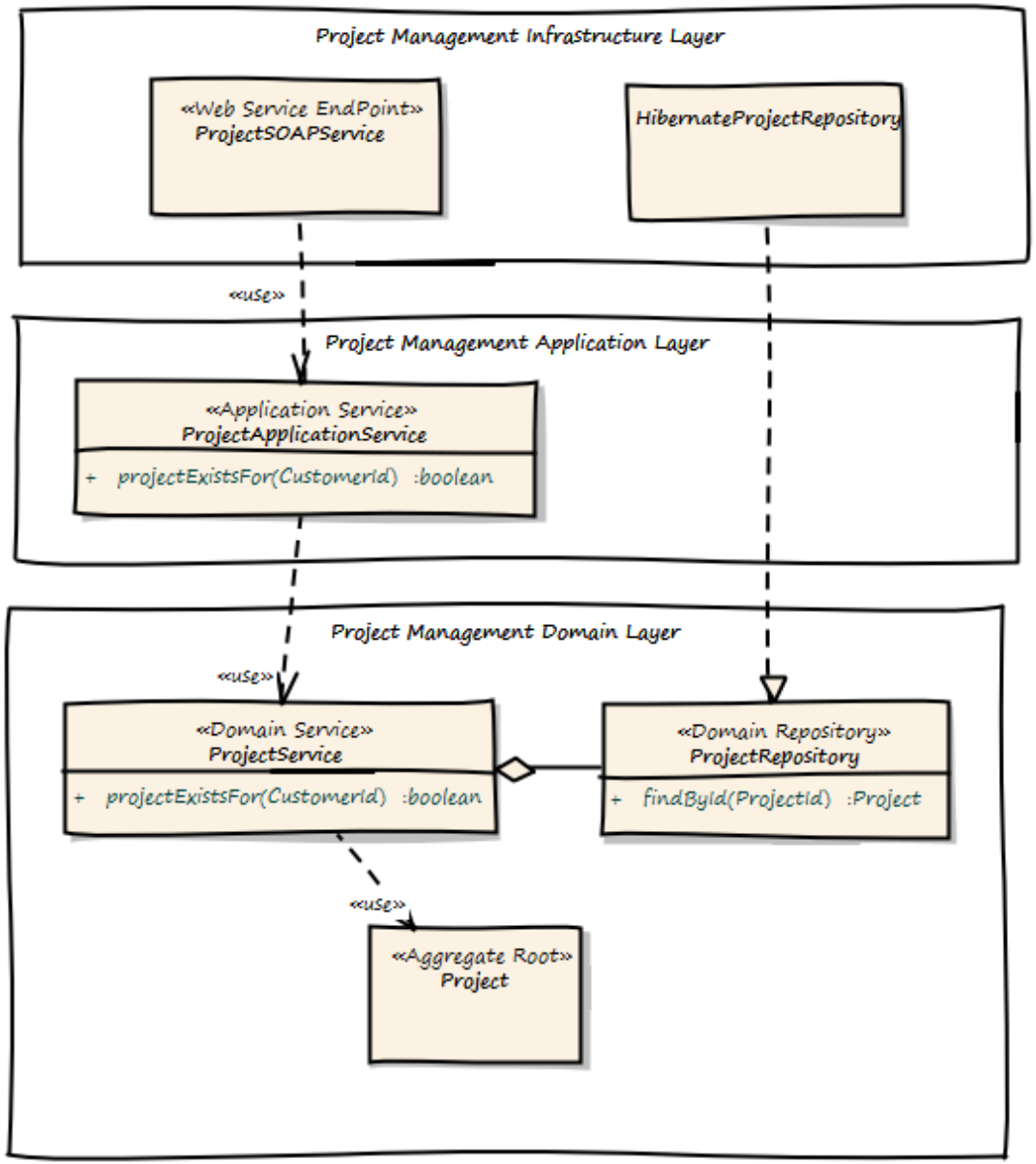
- A Customer can only be deleted if there is no Project assigned

⇒Domain Service

⇒Synchronous Integration



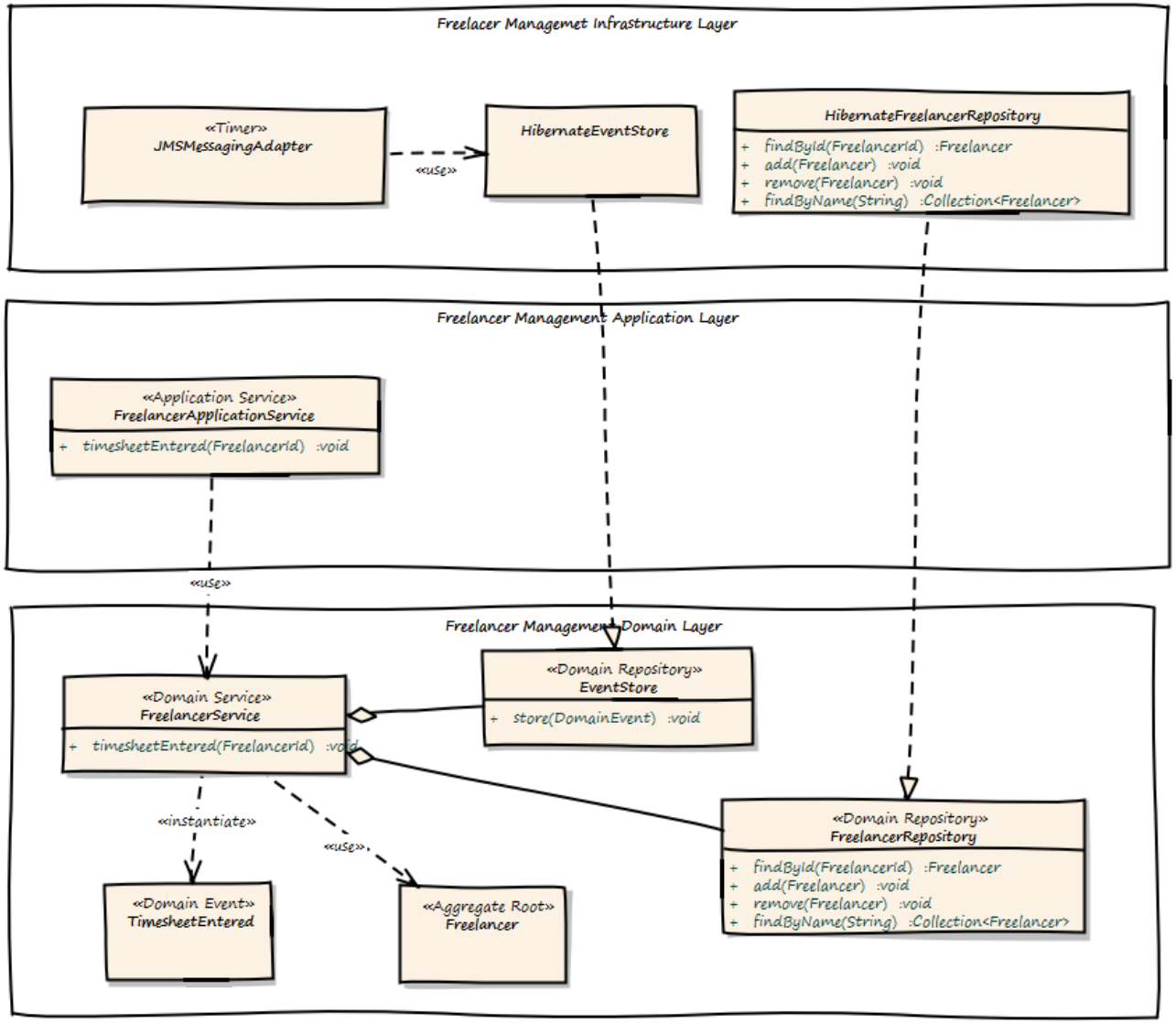
ON THE PROJECT MANAGEMENT SIDE



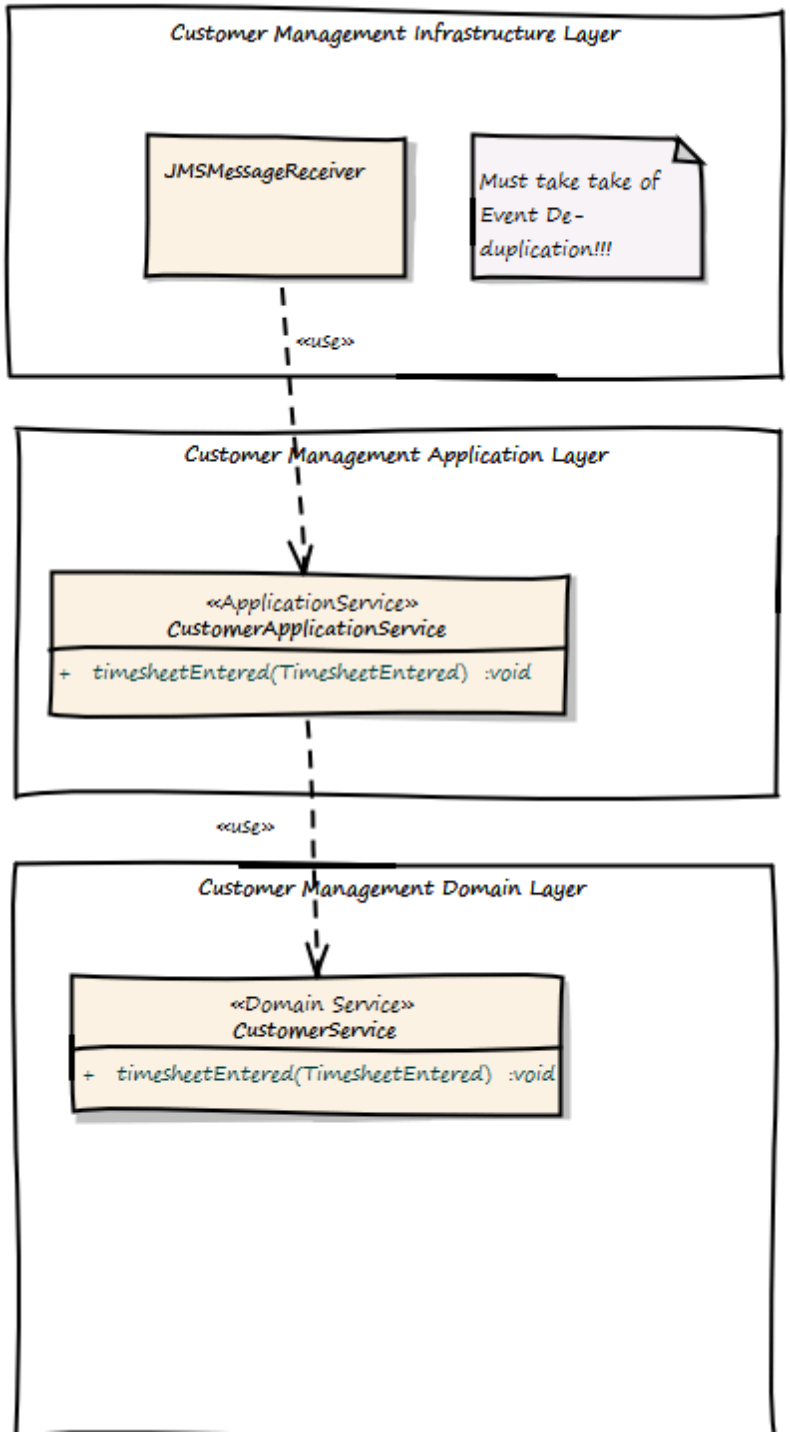
ASYNCHRONOUS EXAMPLE

Once a Timesheet is entered, the Customer needs to be billed

Use Domain Events



ON THE CUSTOMER SIDE



LET'S SWITCH TO MOODLE AND TAKE A QUIZ

It takes 4 minutes.